

第4章 软件架构的风格与模式

王璐璐 wanglulu@seu.edu.cn

廖力 lliao@seu.edu.cn

第4章 软件架构的风格与模式

- 4.0 引言
- 4.1 软件架构风格定义
- 4.2 软件架构风格的分类
- 4.3 典型的软件架构风格
- 4.4 软件架构模式

4.2 软件架构风格的分类

- (1) 数据流风格：批处理、管道/过滤器；
- (2) 调用/返回风格：主程序/子程序、面向对象风格、层次结构；
- (3) 以数据为中心的风格：仓库风格、超文本系统、黑板风格等；
- (4) 虚拟机风格：解释器、基于规则的系统；
- (5) 独立组件风格：进程通讯、事件系统。

4.3 其它软件架构风格

- 1.C2风格
- 2.平台/插件风格
- 3.面向Agent风格
- 4.面向方面软件架构风格
- 5.面向服务架构风格
- 6.正交架构风格
- 7.异构风格
- 8.基于层次消息总线的架构风格
- 9.模型-视图-控制器风格

4.3.10 C2风格

- C2架构风格是一种常见的层次体系架构风格。
- 1995年由California大学Irvine分校的Richard. N. Taylor等人提出
- C2风格的主要思想来源于Chiron-1用户界面系统，因此又被命名为Chiron-2，简称C2。

4.3.10 C2风格

- C2架构风格可以概括为：通过连接件绑定在一起的按照一组规则运作的**并行组件网络**。该规则规定了所有组件之间的交互必须通过**异步消息机制**来实现
- C2是一种基于组件和消息的架构风格，适用于GUI软件开发，构建灵活和可扩展的应用系统。

4.3.10 C2风格

- 其结构如图所示:

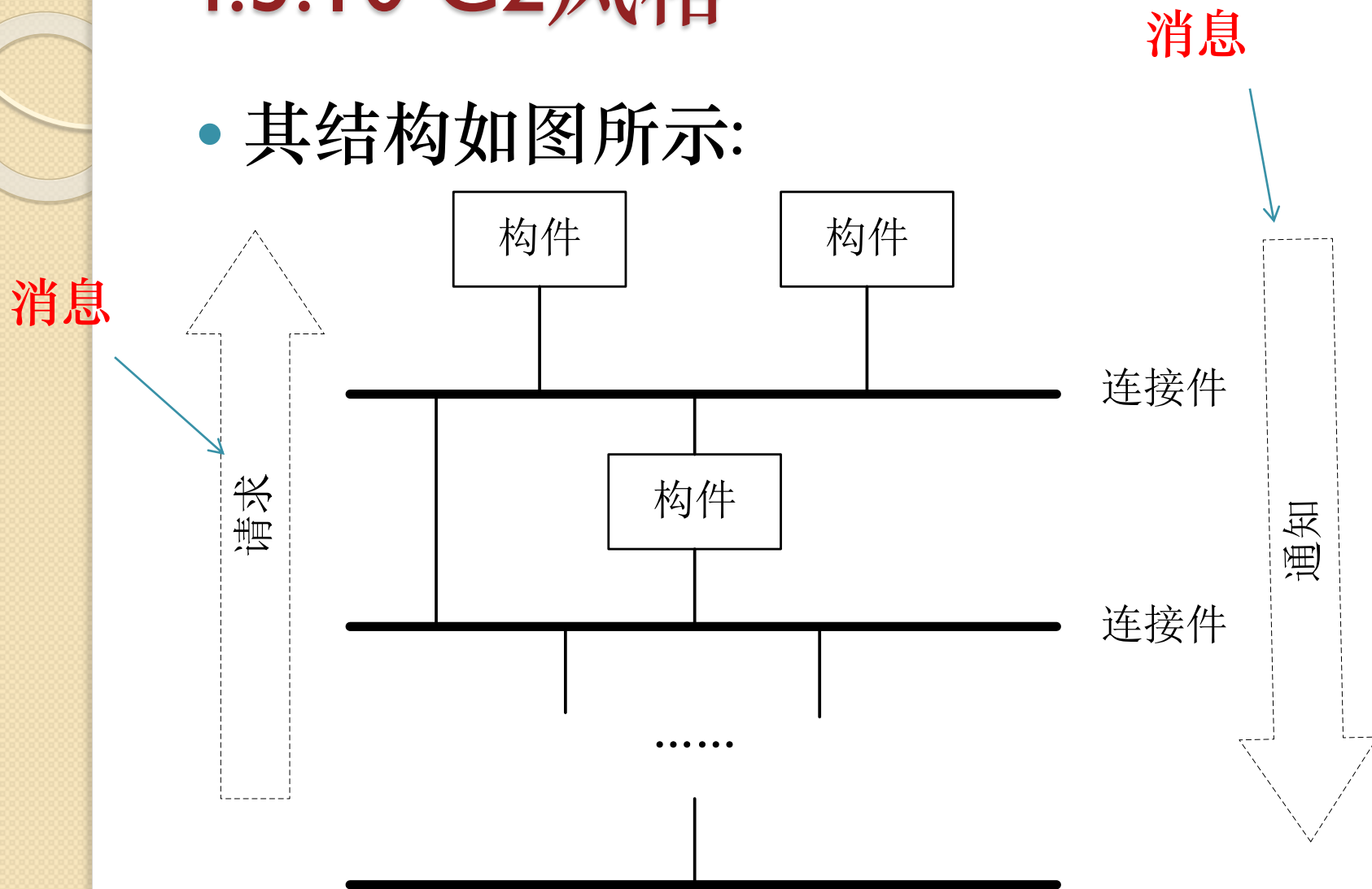


图4.26 C2风格

4.3.10 C2风格

- C2风格的系统组织规则：
 - (1) 组件之间不能直接连接；
 - (2) 组件和连接件都有一个顶部和一个底部；
 - (3) 组件的顶部应连接到某连接件的底部，组件的底部则应连接到某连接件的顶部；
 - (4) 一个连接件可以和任意数目的其他组件和连接件连接；
 - (5) 当两个连接件进行直接连接时，必须由其中一个的底部到另一个的顶部。

4.3.10 C2风格

- C2构件的内部，通信和处理是分开完成的。

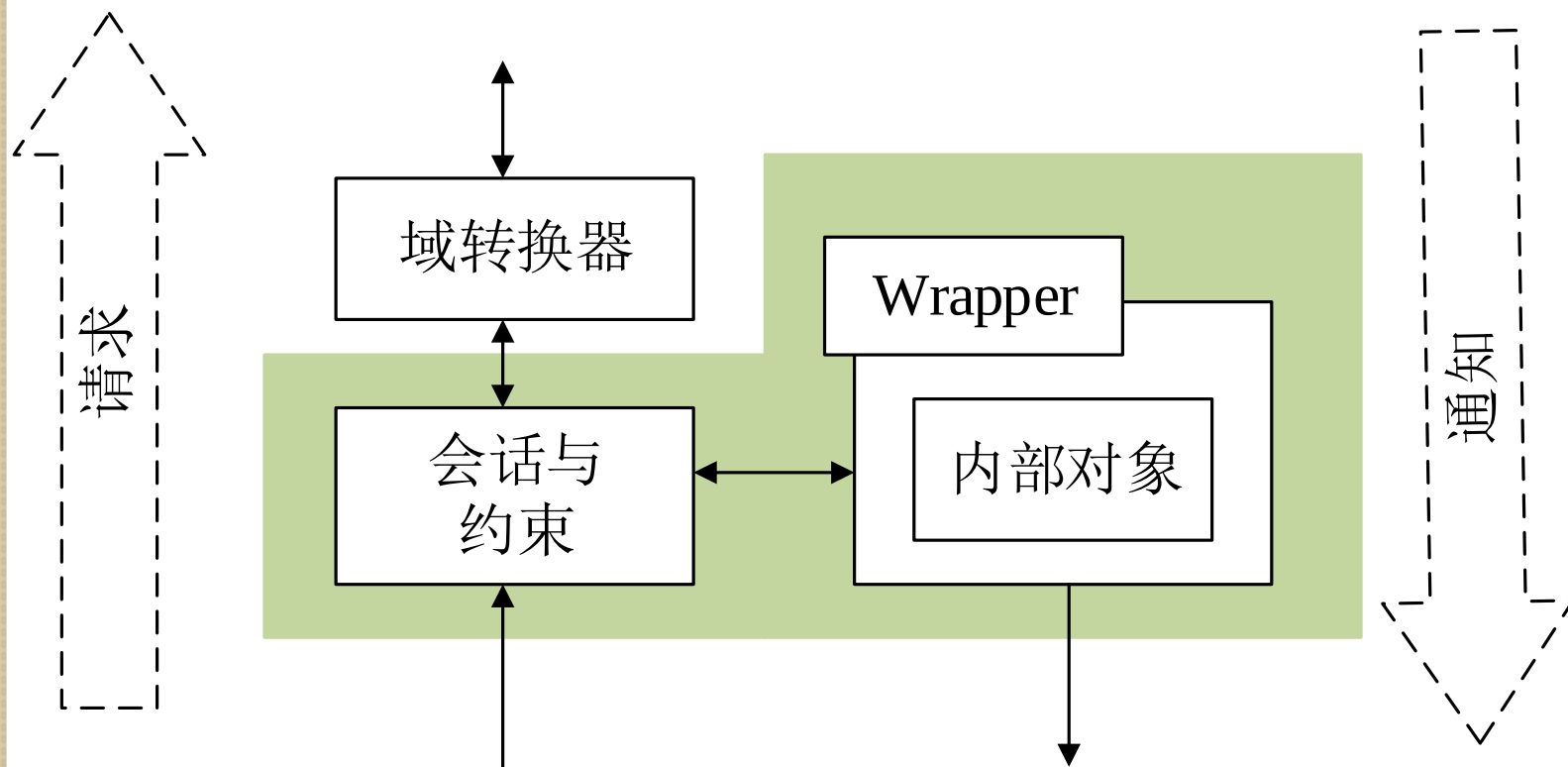
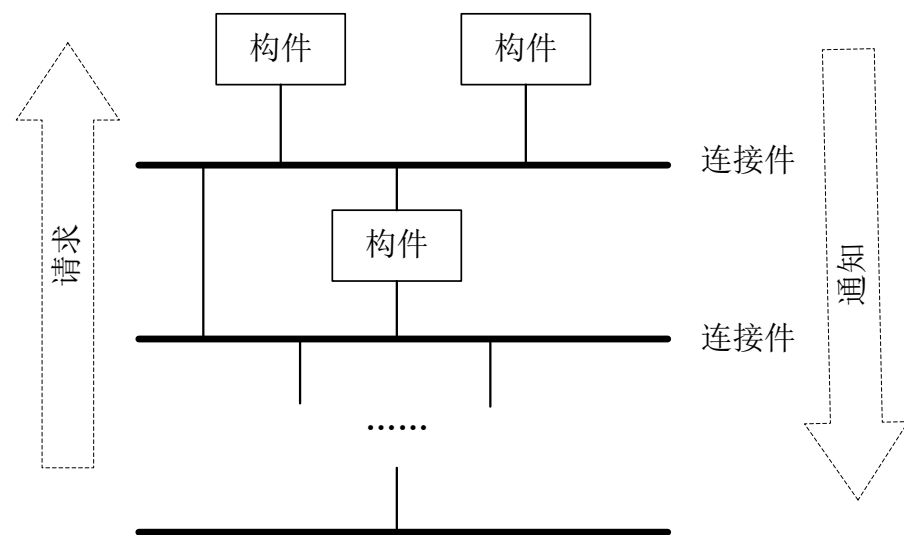


图4-27 用户界面软件的C2组件的内部结构

4.3.10 C2风格

• C2连接件

- 连接件负责把构件绑定在一起,其上可以连接任何数量的构件和连接件。
- 连接件的主要职责
 - 消息的路由和广播
 - 消息的过滤



4.3.10 C2风格

- 优点：

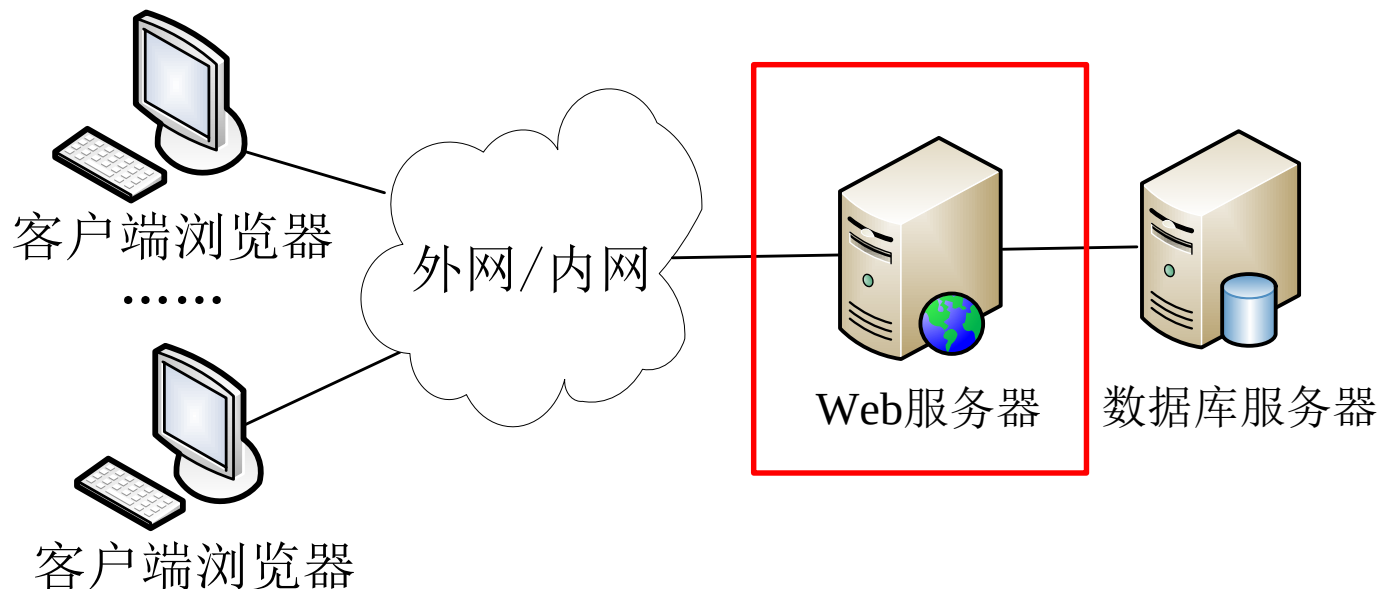
- (1) 可使用任何编程语言开发组件，组件重用和替换易实现；
- (2) 由于组件之间相对独立，依赖较小，因而该风格具有一定扩展能力，可支持不同粒度的组件；
- (3) 组件不需共享地址空间；
- (4) 可实现多个用户和多个系统之间的交互；
- (5) 可使用多个工具集和多种媒体类型，动态更新系统框架结构。

4.3.10 C2风格

- 缺点
 - 不太适合大规模流式风格系统，以及对数据库使用比较频繁的使用。

4.3.10 C2风格

- 应用实例：电能质量报警系统
 - 系统采用B/S三层结构设计（界面层，业务逻辑层，数据访问层），业务逻辑层自动监控报警部分主要采用构件进行处理。



电能质量报警系统

表 5-1 系统 C2 构件与类的对应关系

构件名	构件描述	类名	类说明
DBAI	数据库访问构件	OperationDBBase	数据访问基类
UnitSitePrin	单位、监测点、 负责人构件	Unit	单位类
		Site	监测点类
		Principal	负责人类
EmailSend	邮件发送构件	Email	电子邮件类
ShortMsgSend	短消息发送构件	ShortMsg	手机短消息类
Alarm	报警构件	Alarm	报警类
DevMonitor	设备监控构件	Device	设备类
SagMonitor	电压暂降监控构件	SagEvent	暂降事件类
ComAssMoni	综合评估监控构件	Assessment	综合评估类
GetAndSetAlarm	报警查询与设置构件	AlarmView	报警查看类
		AlarmSet	报警设置类
AlarmService	报警服务构件	PQASService	报警系统服务类

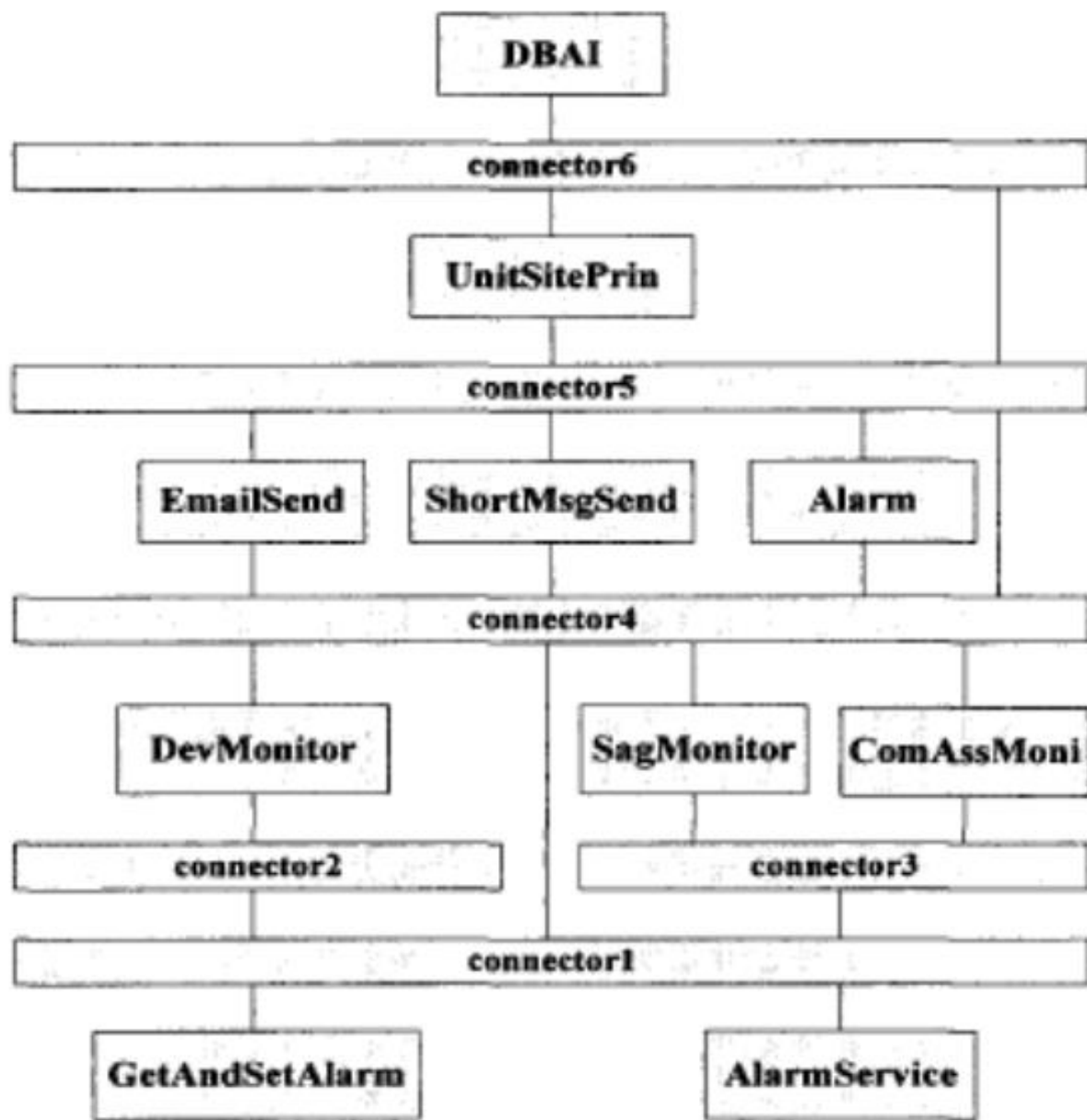


图 5-1 电能质量报警系统的 C2 风格架构

- (1) AlarmService构件读取配置文件获取轮询时间，然后向上层构件发送电压暂降事件监控的请求消息。
- (2) 连接件连接器1收到该消息后，把该消息路由给连接件连接器3。
- (3) 连接件connector3收到消息后，继续进行路由，把消息发送给上层SagMonitor构件。
- (4) SagMonitor构件收到消息后作出响应，向下层构件发送通知消息，最终由connector3和器Connector1接件将该通知消息发送给AlarmService构件。
- (5) SagMonitor构件向上层构件发送从数据库查询暂降事件记录的请求消息，连接件connector4，connector5和connector6将消息路由给DBAI构件.DBAI构件将查询的结果作为通知消息返回给SagMonitor构件。
- (6) SagMonitor构件收到暂降事件记录后，则向上层构件发送消息以判断这些暂降事件是否达到电子邮件报警的级别。该请求消息由connector4连接件路由给报警构件。
- (7) 报警构件收到该消息后，向DBAI构件发送请求以获取用户设定电压暂降电子邮件报警的级别。然后，依此判断是否需要进行电子邮件报警。

- (8) 若暂降事件的幅值和持续时间超过预定的报警级别，则向EmailSend构件发送电子邮件报警的请求。EmailSend构件收到该请求消息后，向UnitSitePrin构件发送请求以获取该事件监测点负责人的电子邮件地址，UnitSitePrin构件把通过请求DBAI构件从数据库中获取到的电子邮件地址信息以通知消息的形式发送给EmailSend构件，然后EmailSend构件以这些电子邮件地址向负责人发送电子邮件告知该电压暂降报警信息。

-
- 版权声明：本文为CSDN博主「qwe3208620」的原创文章，遵循CC 4.0 BY-SA版权协议，转载请附上原文出处链接及本声明。
 - 原文链接：<https://blog.csdn.net/qwe3208620/article/details/80759095>

4.3.10 C2风格

- 应用实例

- KLAX游戏

- 旋转方块，一种街机模拟游戏



游戏逻辑组件

Artist组件

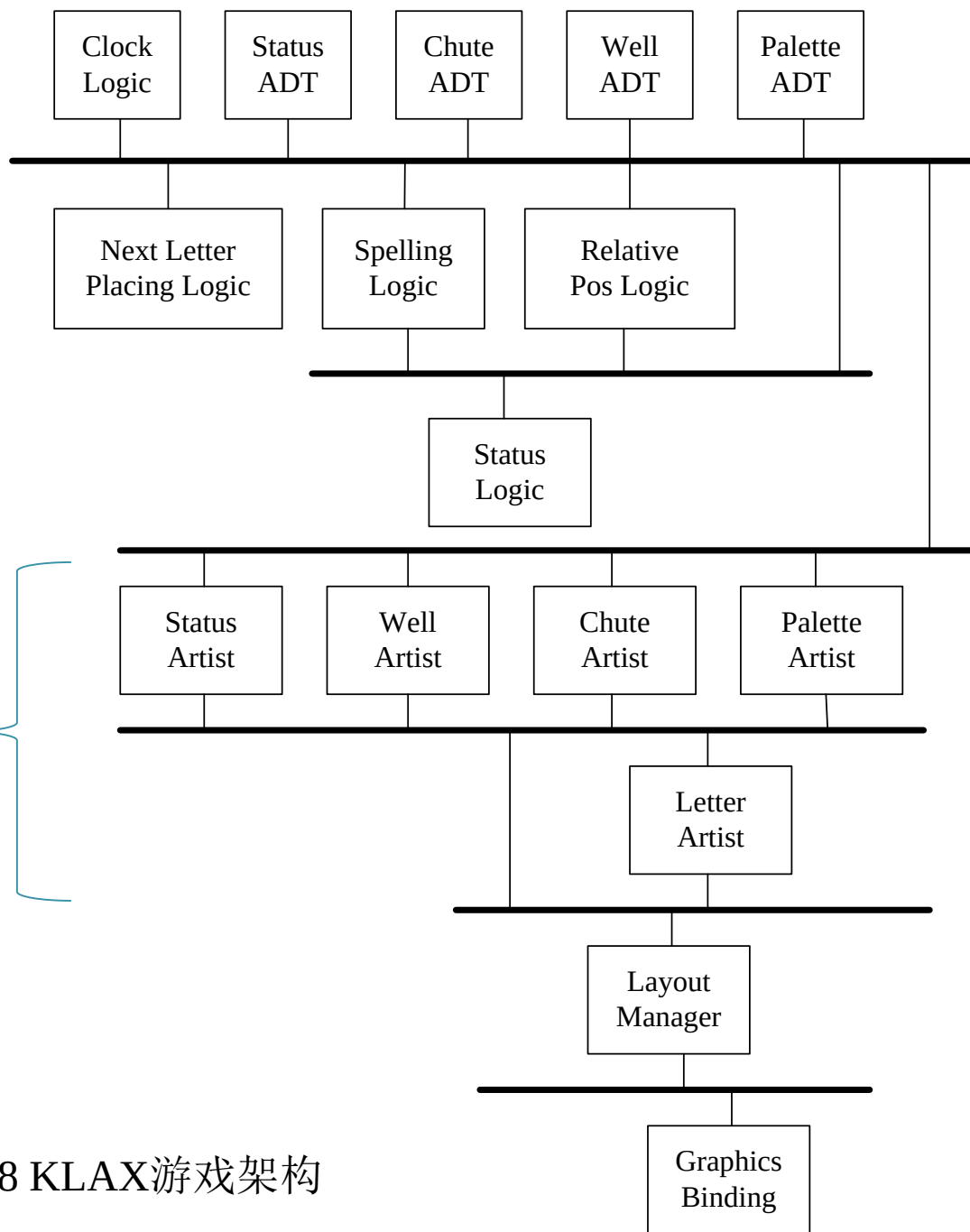


图4.28 KLAX游戏架构

4.3.13 平台/插件风格

电脑管家 - 系统优化

清理垃圾

清除痕迹

插件清理

文件清理

系统加速

发现 44 款插件，清理不需要的插件可提高系统性能

已信任插件(0) 清理记录(2)

插件名称	公司	评价	清理建议	操作
☐ 支付宝安全支付控件 支付宝提供在线安全支付保护功能相关控件。	详情 支付宝	中评	按需要清理	投票 信任
☐ 搜狗拼音语言栏支持模块 搜狗输入法在win7、vista系统下的辅助模块；清理以后不影响输入法...	详情 搜狗	中评	按需要清理	投票 信任
☐ WMP录制组件 Windows系统自带的播放器WindowsMediaPlayer附带的媒体录制...	详情 微软	中评	按需要清理	投票 信任
☐ 百度云管家检测功能控件 百度云管家监测功能相关控件。	详情 百度	中评	按需要清理	投票 信任
☐ 腾讯电脑管家辅助插件 腾讯电脑管家的浏览器辅助插件，提供监控页面、对搜索页嵌入软件...	详情 腾讯	中评	按需要清理	投票 信任

推荐选项

重新扫描

立即清理

4.3.13 平台/插件风格

- 插件（Plug-in）是一种遵循统一的预定义接口规约编写出来的程序，应用程序在运行时通过接口规约对插件进行调用，以扩展应用程序的功能。
- 插件的本质在于不修改程序主体（或者程序运行平台）的情况下对软件功能进行扩展与加强。

4.3.13 平台/插件风格

- 这意味着软件开发者可以通过公布插件的预定义接口规约，从而允许第三方的软件开发者通过开发插件对软件的功能进行扩展，而无须对整个程序代码进行重新编译。

4.3.13 平台/插件风格

- “平台/插件”软件结构将待开发的目标软件分为两部分：
 - 程序的主体或主框架，可定义为平台
 - 功能扩展或补充模块，可定义为插件

4.3.13 平台/插件风格

- 平台所完成的功能应为一个软件系统的核心和基础,可以把平台基本功能分为两个部分:
 - 内核功能：是整个软件的重要功能，一个软件的大部分功能因由内核功能完成。
 - 插件处理功能：用于扩展平台和管理插件，为插件操纵平台和与插件通信提供标准平台扩展接口。
 - 如：Eclipse IDE是Eclipse的运行主体平台，编辑c/c++程序可以应用CDT插件；使用SVN可以安装SVN Repository Exploring。

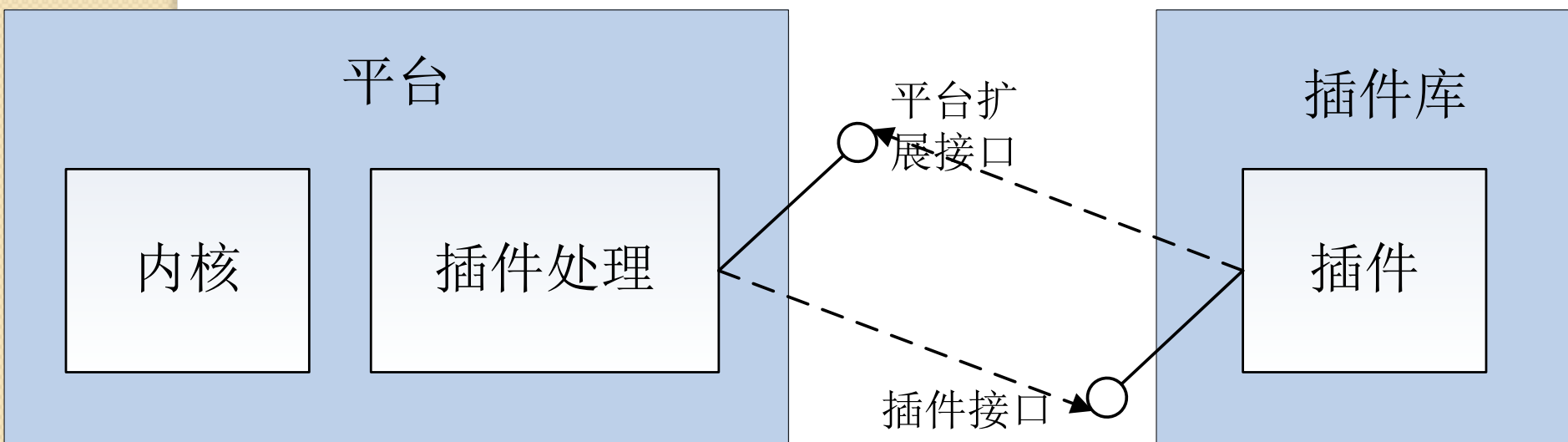
4.3.13 平台/插件风格

- 插件是能动态插入到平台中的程序模块，提供给系统某一方面的功能，但多个插件能使系统功能完善，完成多个复杂的处理。
- 插件受到的约束是：
 - 1) 插件必须能在运行过程中动态地插入平台和从平台中注销，且不影响系统的运行；
 - 2) 当在系统中插入插件后，系统的功能得到扩展或升级；
 - 3) 多个插件之间、插件和平台之间不会发生冲突。

4.3.13 平台/插件风格

- 实现“平台/插件”(Platform / Plug-in) 结构软件需要定义两个标准接口
 - 平台扩展接口：完全由平台实现，插件只是调用和使用
 - 插件接口：完全由插件实现，平台也只是调用和使用。
 - 平台扩展接口实现插件向平台方向的单向通信，插件通过平台扩展接口可获取主框架的各种资源和数据，可包括各种系统句柄，程序内部数据以及内存分配等。
 - 插件接口为平台向插件方向的单向通信，平台通过插件接口调用插件所实现的功能，读取插件处理数据等。

4.3.13 平台/插件风格



4.3.13 平台/插件风格

- 优点：

- (1) 降低系统各模块之间的互依赖性
- (2) 系统模块独立开发、部署、维护
- (3) 根据需求动态的组装、分离系统

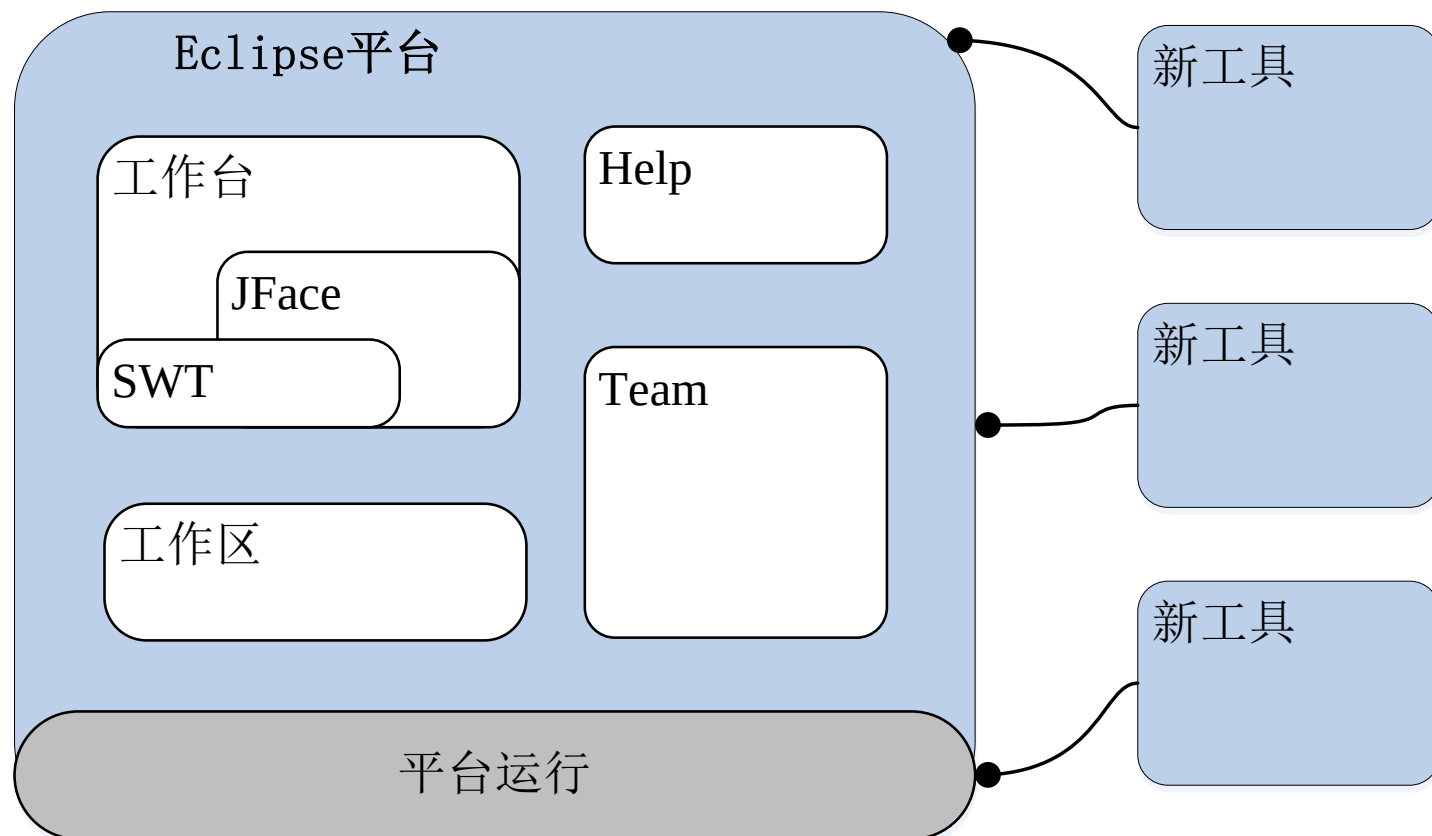
- 缺点

- 插件是别人开发的可以用到某主程序中的，只服务于该主程序，可重用性差。

4.3.13 平台/插件风格

- 应用实例

- Eclipse开发平台



4.3.14 面向Agent风格

- 面向Agent风格的一个重要特点，就是编程时不再仅仅只是用对象的思维来考虑问题，而是要更进一步，用Agent智能体的思维去考虑问题。

4.3.14 面向Agent风格

- 面向Agent风格（Agent-oriented）的基本思想是认为事物的属性，特别是动态特性在很大程度上受到与其密切相关的人和环境的影响，将影响事物的主观与客观特征相结合抽象为系统中的Agent，作为系统的基本构成单位，通过Agent之间的合作实现系统的整体目标。

4.3.14 面向Agent风格

- Agent: 一个能够根据它对其环境的感知，主动采取决策和行为的软件实体。
- Agent组件: 对系统处理的高度抽象，具有高度灵活和高度智能特色的软件实体。
- Agent连接件: 对复合型组件的连接，该连接能够提供通信、协调、转换、接通等服务。

4.3.14 面向Agent风格

- Agent组件有别于以往任何系统的组件类型，其所具有的自主性、智能性、交互性等特性是传统架构对象所不具备的。
- 多Agent系统中的连接件并非显式地将两个不同的组件联系起来。不同Agent之间的联系是根据运行时状态来决定的。

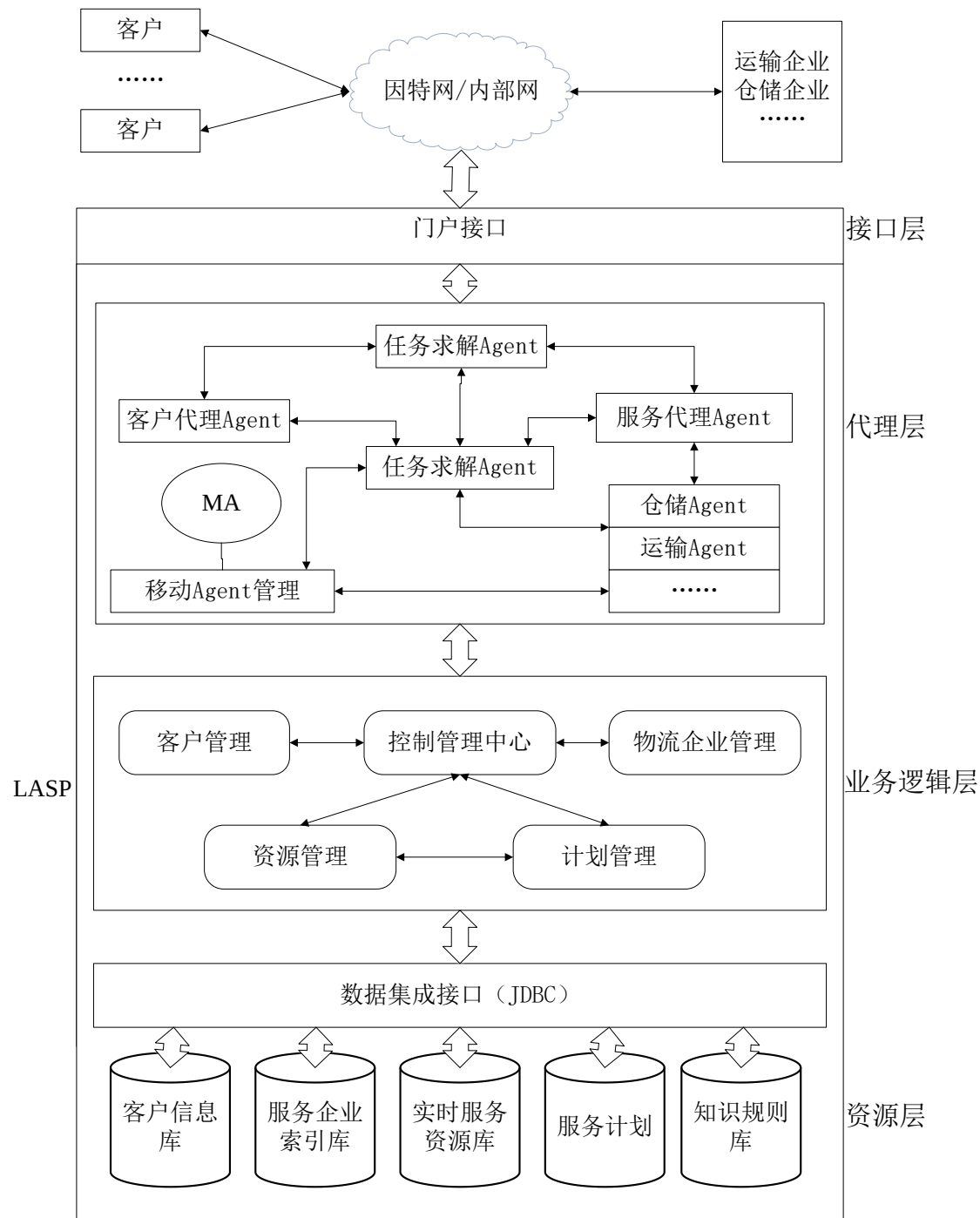


图4-38LASP物流管理系统的架构

4.3.14 面向Agent风格

- 优点

- 面向Agent的软件工程方法对于解决复杂问题是一种好的技术,特别是对于分布开放异构的软件环境。

- 缺点

- 大多数结构中Agent自身缺乏社会性结构描述和与环境的交互。

4.3.15 面向方面软件架构风格

- 面向方面的编程(AOP: Aspect Oriented Programming)
 - 一般认为AOP在传统软件架构基础上增加了方面组件(Aspect Component)这一新的构成单元,通过方面组件来封装系统的横切关注点。
 - 系统的有些特性和需求是横切于系统的每一个层面中,并融于系统的每一个组件中,这种特性称为系统的方面(Aspect)需求特性或关注点
 - 如系统中的时间要求、业务逻辑、性能、安全性和错误检测、QoS监测等。

4.3.15 面向方面软件架构风格

- 应用AOP的主要目的----尽量分离“技术问题实现”和“业务问题实现”
 - 它允许开发者能够对横切关注点进行模块化设计；
 - 能够实现分散关注，将通用需求功能从不相关类之中分离出来；
 - 能够实现代码重用。

4.3.15 面向方面软件架构风格

- 例如：日志功能
 - 日志代码往往水平地散布在所有对象层次中----在控制层、业务层和数据访问层都需要日志功能，并且而与它所散布到的对象的核心功能毫无关系。

4.3.15 面向方面软件架构风格

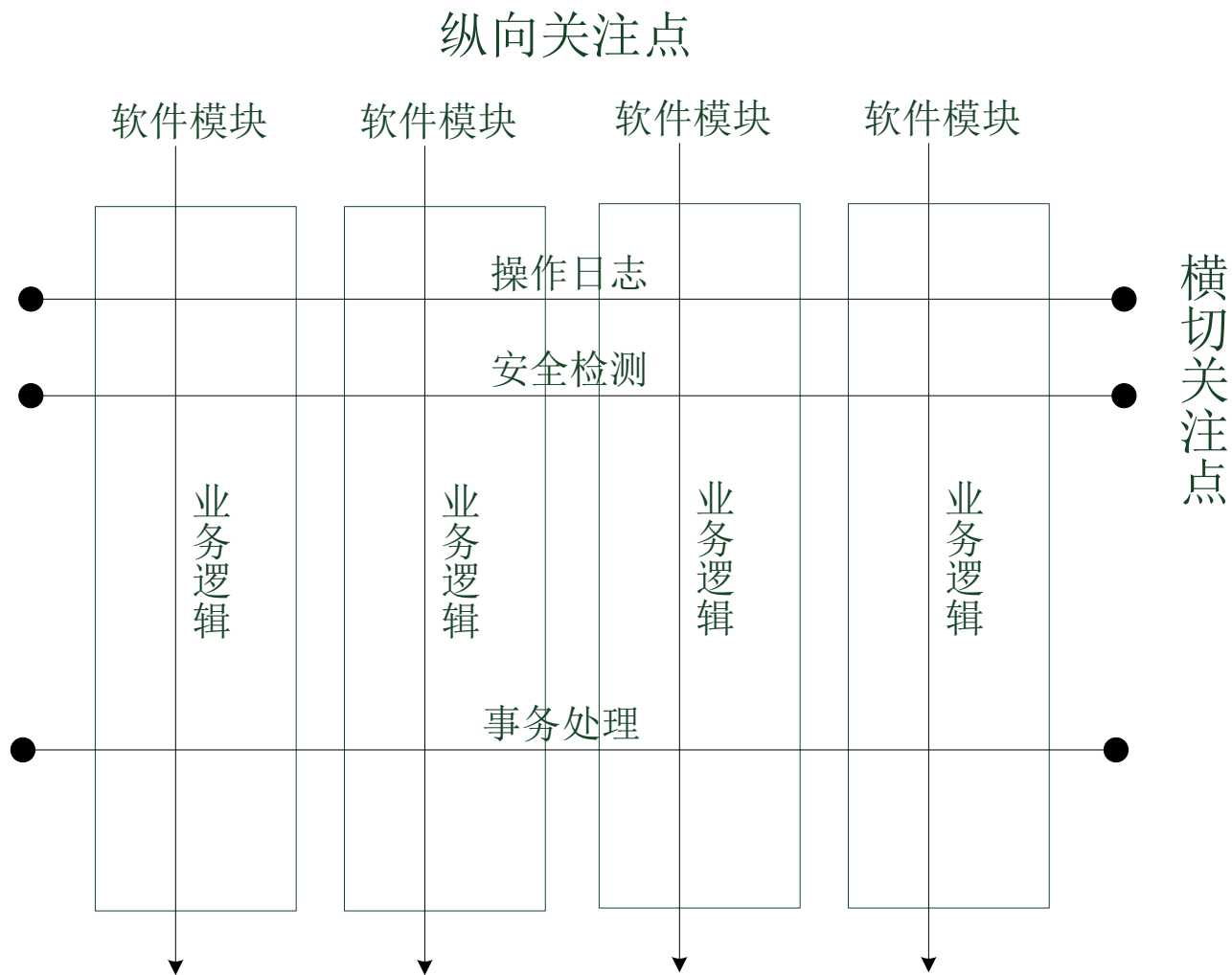
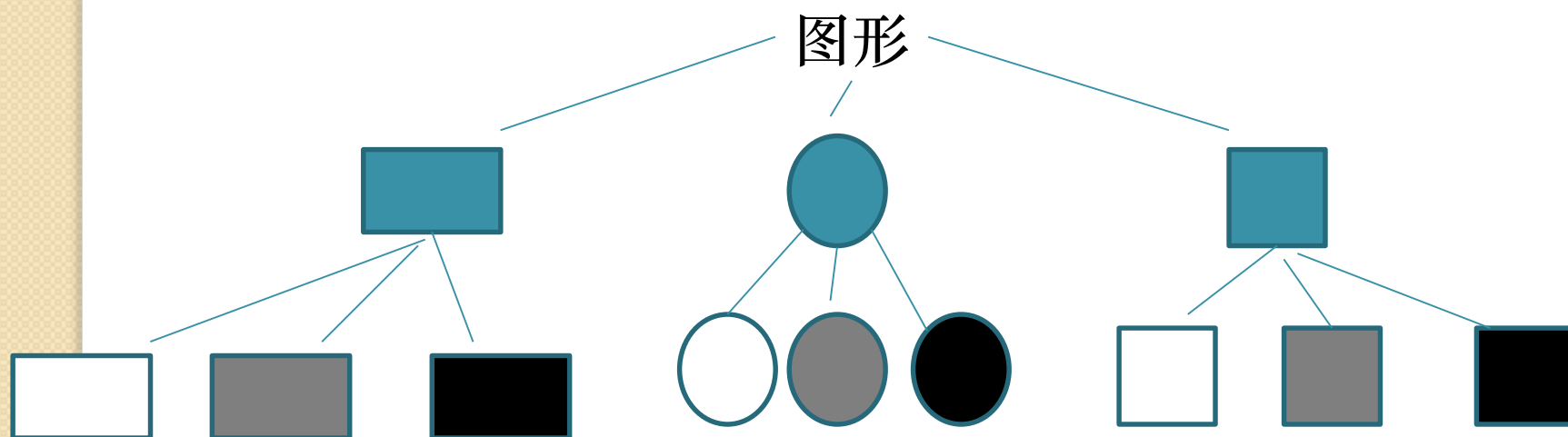


图4.39 AOP示意图

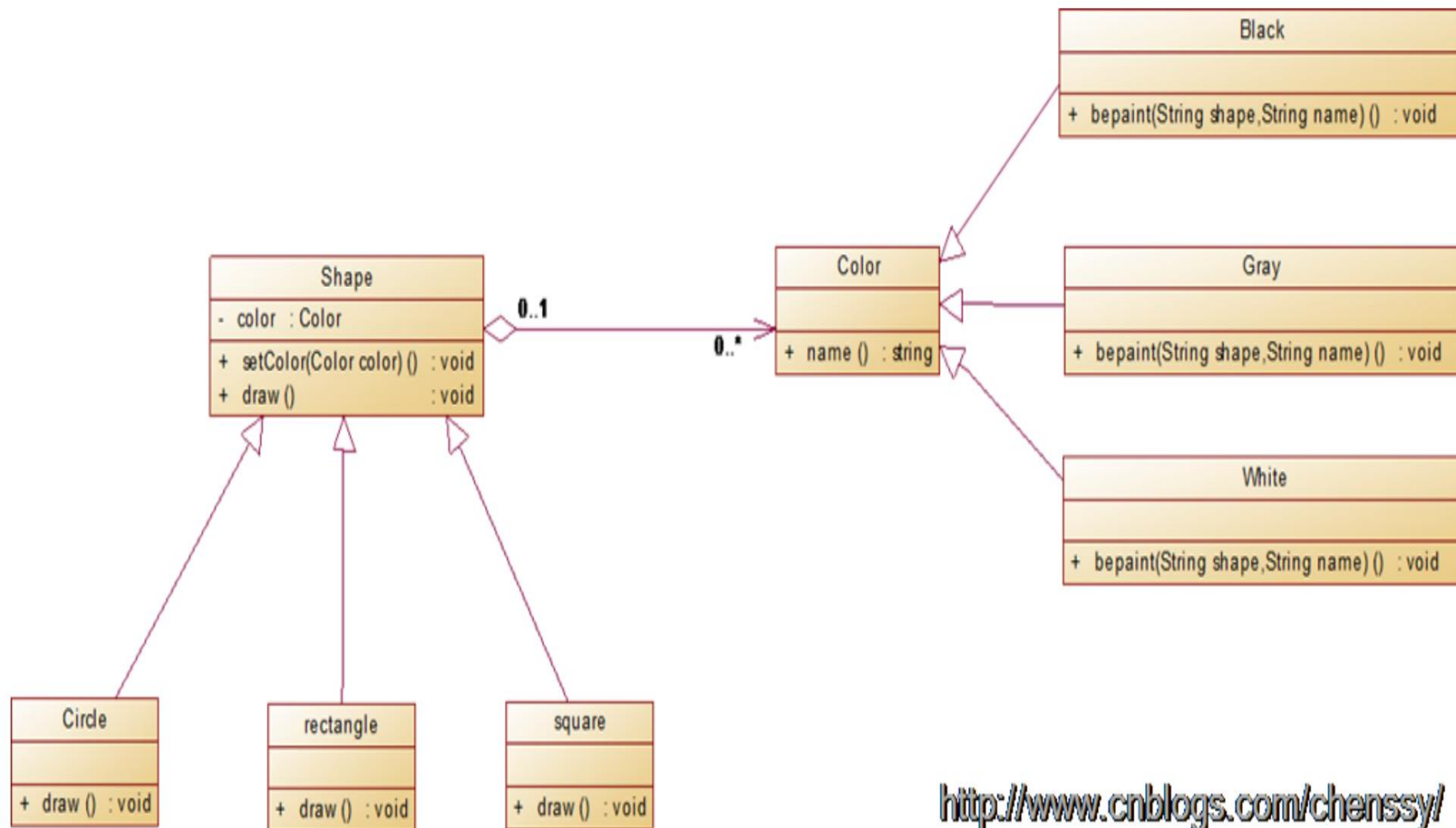
例子

- 画图，这里有一个画笔，可以画正方形、长方形、圆形，现在需要给这些形状进行上色，这里有三种颜色：白色、灰色、黑色。这里我们可以画出 $3*3=9$ 中图形



例子

模式场景我们就采用哪个画图的。其UML结构图如下：



4.3.15 面向方面软件架构风格

- 优缺点分析

- 可以定义交叉的关系，并将这些关系应用于跨模块的、彼此不同的对象模型
- AOP 同时还可以让我们层次化功能性而不是嵌入功能性，从而使得代码有更好的可读性和易于维护。
- 它会和面向对象编程可以很好地合作，互补

4.3.16 面向服务架构风格

- 1996年，Gartner Group 提出面向服务架构模型SOA
- SOA 是一个组件模型，它将应用程序的不同功能单元（服务）通过这些服务之间定义良好的接口和契约联系起来。
 - 服务(service)是一个粗粒度的、可发现的软件实体。
 - 接口是采用中立的方式进行定义的，应独立于实现服务的硬件平台、操作系统和编程语言，便于不同系统中的服务以一种统一和通用的方式进行交互。

4.3.16 面向服务架构风格

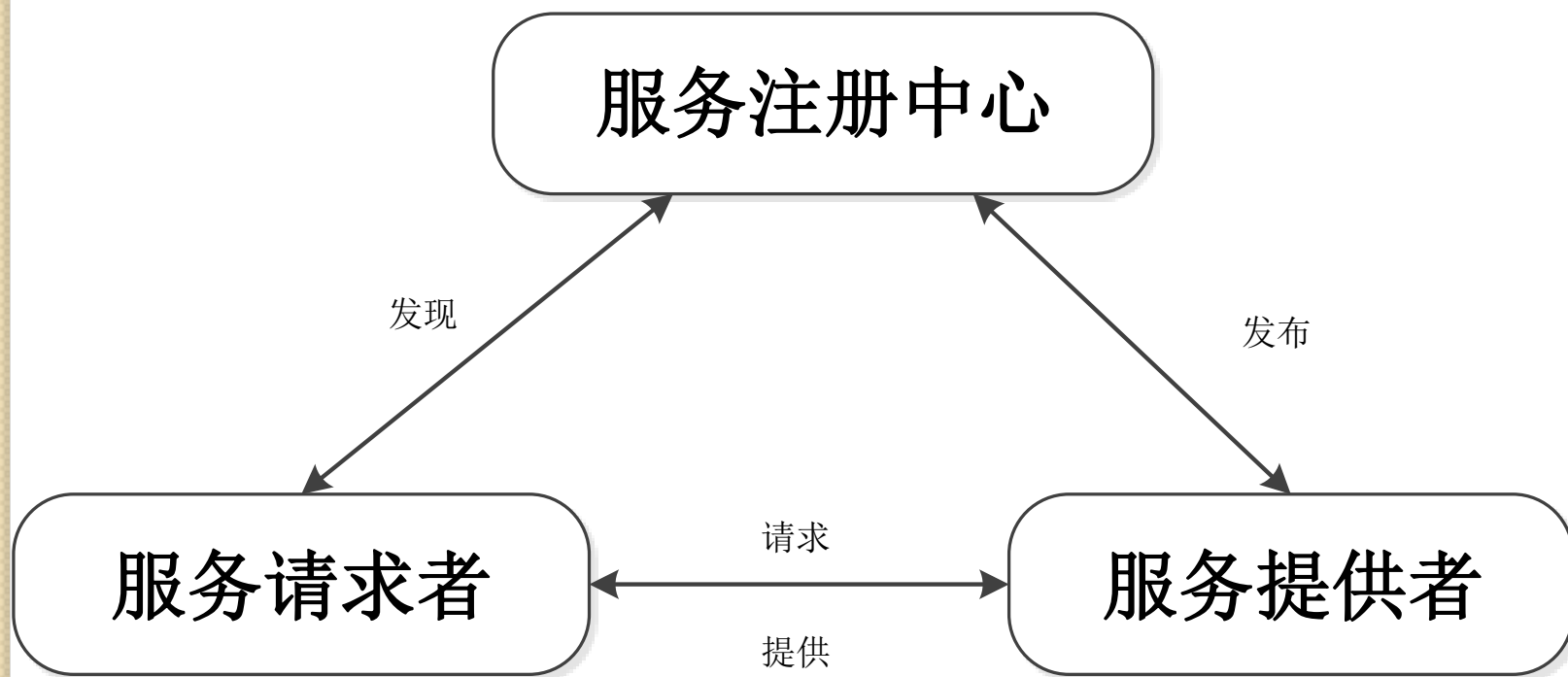


图4.41 SOA架构

4.3.16 面向服务架构风格

- **服务请求者**：可以是服务或者第三方的用户，通过查询服务提供者在服务注册中心发布的服务接口的描述，通过服务接口描述来通过RPC或者SOAP进行绑定调用服务提供者所提供的业务或服务。
- **服务提供者**：作为服务管理者和创建者，必须将服务描述的接口发布到服务注册中心才能被潜在的服务请求发现，能够为合适的服务请求者提供服务。
- **服务注册中心**：相当于服务接口的管理中心，服务请求者能够通过查询服务注册中心的数据库来找到需要的服务调用方式和接口描述信息。

4.3.16 面向服务架构风格

- **发布**：为了便于服务请求者发现，服务提供者将对服务接口的描述信息发布到服务注册中心上。
- **发现**：服务请求者通过查询服务注册中心的数据库来找到需要的服务，服务注册中心能够通过服务的描述对服务进行分类，使服务提供者更快定位所需要的服务范围。
- **绑定和调用**：服务请求者在查询到所需要服务描述信息，根据这些信息服务请求者能够调用服务。

4.3.16 面向服务架构风格

- 例如：假设一个服务集成者现在需要开发一个组合服务，该服务的基本功能为分析当前用户的IP地址，然后获得用户所在城市的天气预报。
- 可分解为如下服务：
 - 从IP地址获得城市名称的服务
 - 从城市名称获得邮政编码的服务
 - 从邮政编码获得天气服务

4.3.16 面向服务架构风格

- 解决方案：

- 通过Seekda进行搜索

- Seekda是一个专用Web服务搜索引擎，2016年左右涵盖了全球范围内7739个服务提供者提供的28606个Web服务，该搜索引擎返回的结果包含服务地址，服务接口文档信息和服务提供商信息等。

- 搜索结果

- IP + city 99个
 - city + zipcode 66个
 - zip + weather 17个

4.3.16 面向服务架构风格

- 解决方案：
 - 服务过滤器将不符合特定要求的具体服务进行筛选和过滤，该组织分三个步骤对服务进行过滤：
 - 服务不含有能够提供需求功能的操作，即服务不能提供其所称的功能；
 - 服务末端URI地址不存在或服务的信任值低于阈值；
 - 服务不具备良好定义格式或包含错误的WSDL文档。
 - 筛选后得到：
 - IP + city 8个
 - city + zipcode 4个
 - zip + weather 3个

4.3.16 面向服务架构风格

- 解决方案：
 - 最终通过各组合方案的质量属性对比可以确定最后的组合方案。

表 4.4 候选绑定方案及对应的各维 QoS 属性值

编号	绑定方案			执行时间/ms	价格/yuan	可靠性/%	可用性/%	声誉/[0, 1]
1	CS ₀₁	CS ₁₂	CS ₂₁	1168	942	48.45	65.10	0.86
2	CS ₀₁	CS ₁₂	CS ₂₂	1557	657	38.18	58.67	0.87
3	CS ₀₁	CS ₁₃	CS ₂₁	944	863	49.08	57.92	0.79
4	CS ₀₁	CS ₁₃	CS ₂₂	1333	578	38.67	52.20	0.80
5	CS ₀₄	CS ₁₂	CS ₂₁	1463	977	54.11	60.85	0.84
6	CS ₀₄	CS ₁₂	CS ₂₂	1852	692	42.63	54.85	0.85
7	CS ₀₄	CS ₁₃	CS ₂₁	1239	898	54.81	54.14	0.77
8	CS ₀₄	CS ₁₃	CS ₂₂	1628	613	43.19	48.80	0.78
9	CS ₀₇	CS ₁₂	CS ₂₁	1673	1104	60.81	69.67	0.90
10	CS ₀₇	CS ₁₂	CS ₂₂	2062	819	47.92	62.79	0.91
11	CS ₀₇	CS ₁₃	CS ₂₁	1449	1025	61.60	61.98	0.82
12	CS ₀₇	CS ₁₃	CS ₂₂	1838	740	48.54	55.87	0.83



麻雀商业评论

2022-4-18 01:09 · 互联网行业观察者

上海有个群体被大家忽略了，困在上海不懂中文又要靠微信群维生的外国友人们，整天被微信翻译搞得心惊肉跳，估计现在急需心理疏导和靠谱翻译服务。

Group Notice

各位宝宝们下午好~

明天下午1点公寓会安排消杀工作，如果有需要入户消杀的宝宝们请微信与我们沟通哈

Good afternoon.

Tomorrow afternoon at 1: 00 apartment will arrange the the killing work, if there is a need to kill the baby in the house, please communicate with us through WeChat

Translated by WeChat

头条 @清茶520端



长河国际公寓客服部

各位住客 今日消杀已完毕

Translated by WeChat

Ladies and gentlemen, the killing is over today.

头条 @清茶520端

头条 @清茶520端

大家注意防护，有消息说很多小区是团菜团出很多病例，大家一个个下楼注意间距

We pay attention to protection, there is news that a lot of small groups is the group of dishes out a number of cases, we go downstairs one by one attention spacing

Translated by WeChat

1:41 PM

回去迅速消杀，自己做好消杀

Go back and kill quickly. Kill yourself.

Translated by WeChat

头条 @清茶520端

回去迅速消杀，自己做好消杀

Go back and disinfect quickly. Do it yourself.

Translated by Weixin

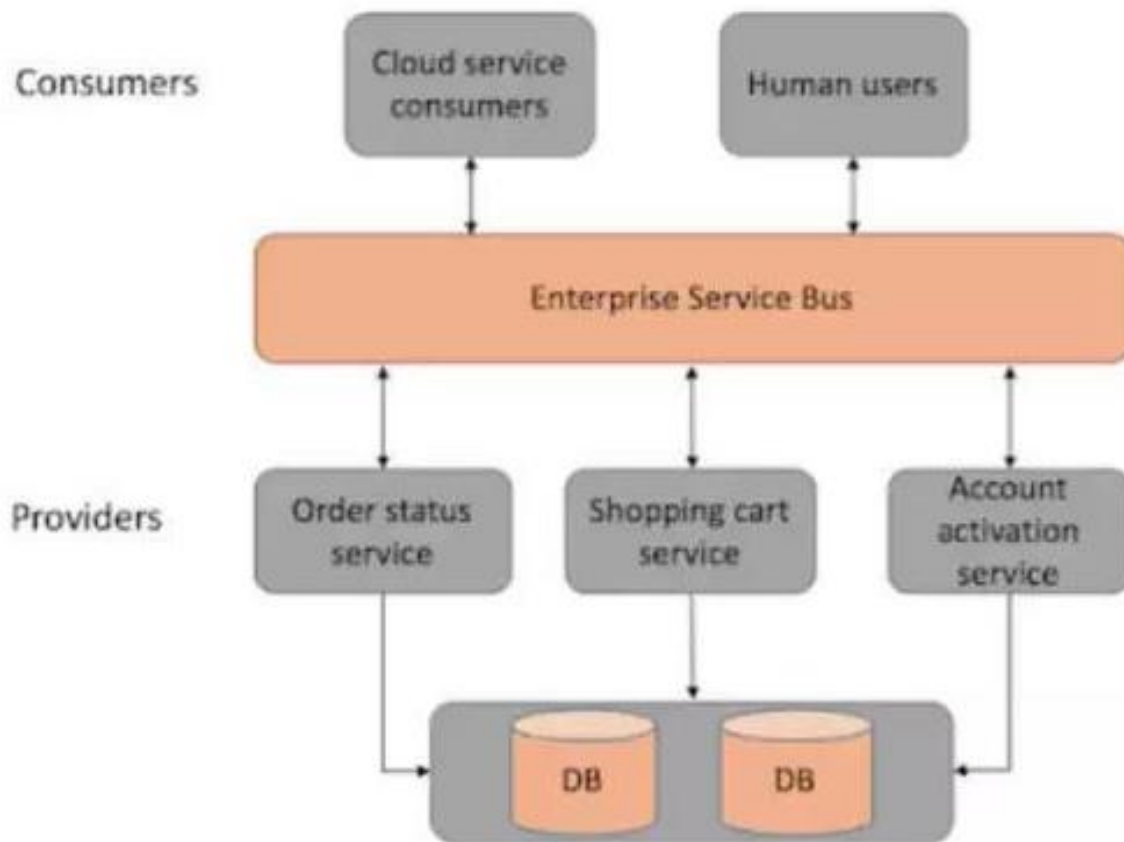


4.3.16 面向服务架构风格

- 面向服务软件架构风格在于具有基于标准、松散耦合、共享服务和粗粒度等优势，表现为易于集成现有系统、具有标准化的架构、提升开发效率、降低开发维护复杂度等。
- 通过采用SOA架构，在进行一次开发成本急剧减少的同时，由于系统具有松散耦合的特征使得维护成本也大大减少。

ESB和SOA

- SOA的最常见的实现方式方式是SCA和JBI两个标准，而SCA的实现需要ESB，JBI则不需要ESB。



4.3.16 面向服务架构风格

- 应用实例
 - 金蝶EAS
(Enterprise Application Suite)

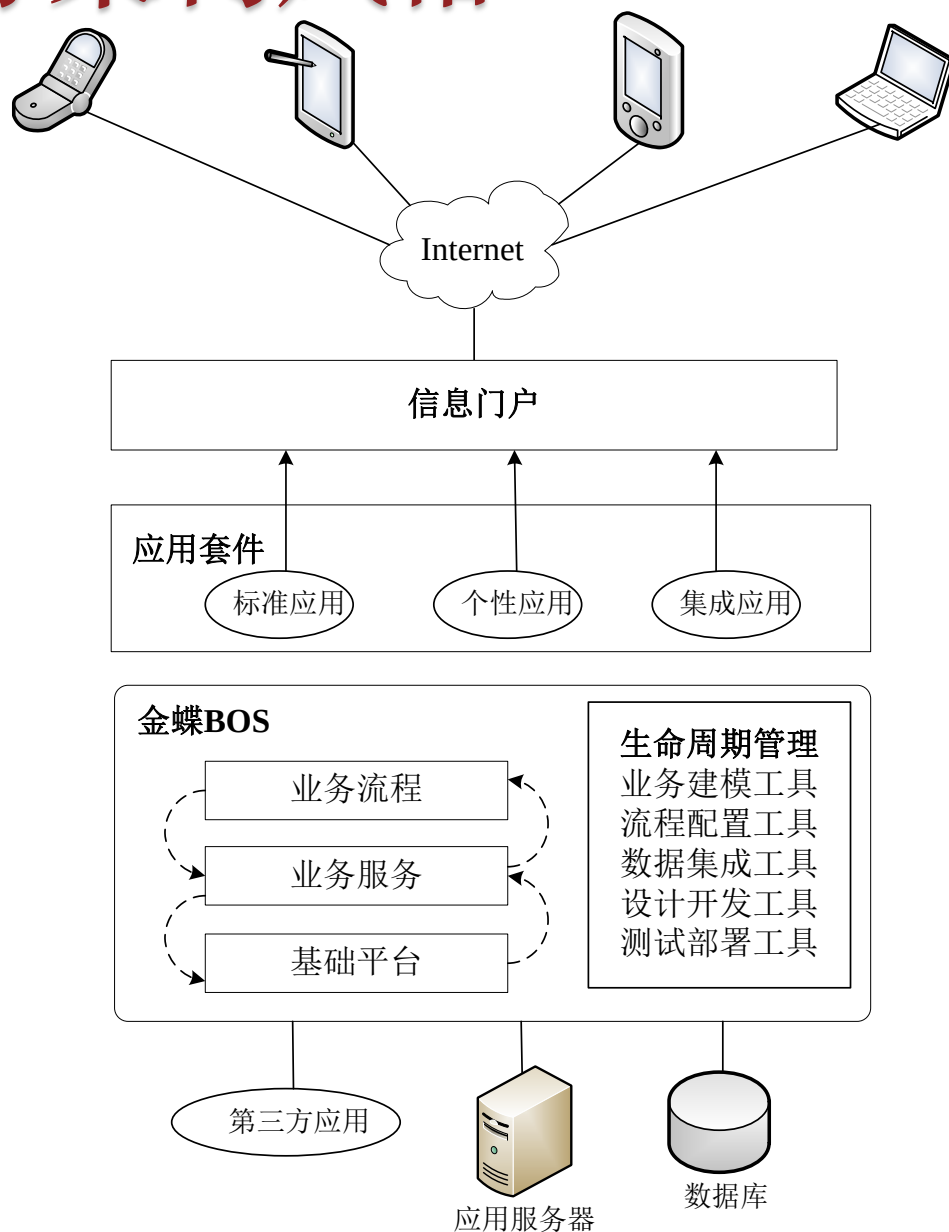


图4.42金蝶EAS技术架构

4.3.16 面向服务架构风格

- 优点

- (1) 灵活性，根据需求变化，重新编排服务。
- (2) 对IT资产的复用。
- (3) 使企业的信息化建设真正以业务为核心。业务人员根据需求编排服务，而不必考虑技术细节。

4.3.16 面向服务架构风格

- 缺点

- (1) 服务的划分很困难。
- (2) 服务的编排是否得当。
- (3) 如果选择的接口标准有问题，如主流的Web service之类，会带来系统的额外开销和不稳定性。
- (4) 对IT硬件资产还谈不上复用。
- (5) 目前主流实现方式接口很多，很难统一。
- (6) 目前主流实现方式只局限于不带界面的服务的共享。

微服务架构



With monolithic, tightly coupled applications, all changes must be pushed at once, making continuous deployment impossible.



Traditional SOA allows you to make changes to individual pieces. But each piece must be carefully altered to fit into the overall design.



With a microservices architecture, developers create, maintain and improve new services independently, linking info through a shared data API.

微服务架构

- 微服务架构是面向服务架构的一个变形。
- 微服务架构（Microservice Architect）是一种将单个应用程序开发为一组小型服务的方法，每个小型服务都在自己的进程中运行，并以轻量级机制（通常是HTTP资源API）进行通信。
- 这些服务围绕业务功能构建，可通过全自动部署机制独立部署。
- Amazon、Netflix、LinkedIn、Spotify、SoundCloud和其他公司已将其应用架构演变为微服务架构（MSA）。

微服务架构

- 微服务架构风格的特点是：
 - 围绕业务能力组织系统；
 - 自动化部署；
 - 智能化；
 - 对程序和数据的去中心化控制。
- 这种风格可以设计出灵活、模块化且易扩展的架构。

微服务架构

- 微服务架构是面向服务架构的一个变形，由于它注重于单个服务的构建，能够轻松的增加、改进或删除服务，因此具有**松耦合**的特点。
- 使用微服务进行架构设计并非易事，因为使用微服务意味着需要**管理分布式架构**及接受其所带来的一**系列挑战**，包括管理网络延迟和不可靠性、容错能力、复杂服务的编排、数据一致性、事务管理以及负载均衡。
- 云基础架构和新技术在实现微服务架构以及克服相关挑战和复杂性方面发挥着重要作用。

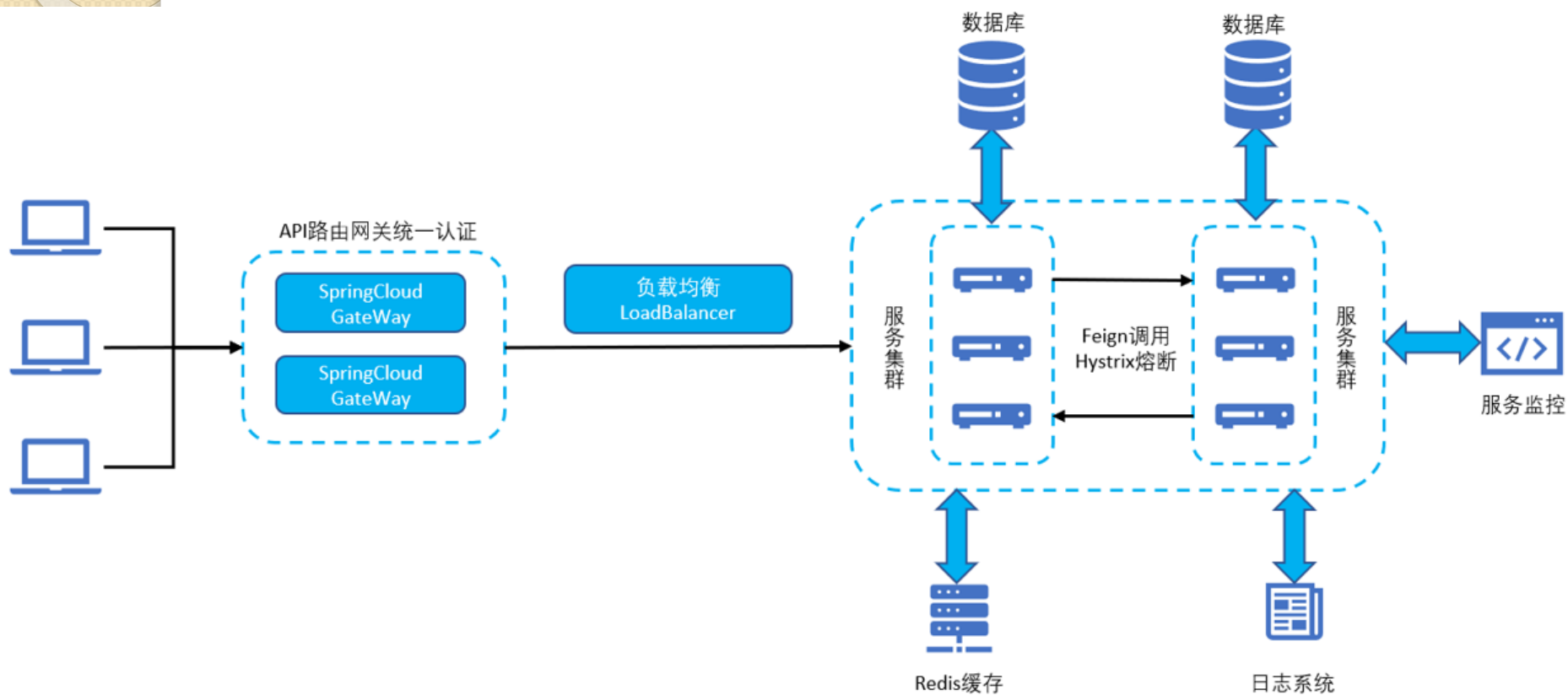
示例：SpringBlade微服务平台

- SpringBlade 是由一个商业级项目升级优化而来的SpringCloud微服务架构；
- 采用Spring Boot 2.4 、 Spring Cloud 2020、Mybatis 等核心技术，用于快速搭建企业级的SaaS微服务系统平台。
- 它提供一个可供快速使用及扩展的**公司人员及事务管理系统平台**。

示例：SpringBlade

- 功能总体说明：
 - 工作台：提供通知公告的增、删、改、查功能。
 - 事务管理：处理企业人员的各种审批事务
 - 报表管理：定制及发布公司报表
 - 权限管理：对角色、数据、接口进行权限配置
 - 系统管理：对系统的界面、各项功能、人员进行管理
 - 资源管理：对一些资源（空间、附件等）进行管理
 - 流程管理：对事务模块的处理流程进行定义与管理
 - 基础配置：对企业需要用到的社会常识信息进行配置
 - 研发工具：对系统业务及数据进行扩展。
 - 系统监控：对整个系统进行监控、运维

示例：SpringBlade



示例：SpringBlade

- **学生实验**：基于SpringBlade自己搭建了3个微服务bookApplication, BorrowApplication, UserApplication；3个微服务可以与各自的数据库交互，BorrowApplication可以远程调用bookApplication, UserApplication完成借书操作。
- **实验发现**：实现远程调用，微服务就有了最基础的框架，多个微服务就可以协同完成一项任务。

微服务架构

- 优点：

- 每个微服务模块都是独立部署独立维护的，每次更新都不会对整体业务有所影响，而且是基于共享的通讯机制的。
- 在部署或者扩容的时候并不需要对整个应用进行整体扩容，只需要将应用中遇到性能瓶颈的模块进行扩容就可以了，这样的更细粒度的扩容方式可以节省资源。

微服务架构

- 缺点：
 - 分布性系统天生的复杂性，如网络延迟、容错性、不可靠的网络、异步机制等
 - 运维开销及成本增加，一个单体应用被拆分部署成十几个服务，增大了运维的压力。
 - 隐式接口及接口匹配问题，不同组件间协作需要统一可管理的接口。
 - 代码重复，为了尽量实现松耦合，相同功能的底层功能会在不同服务内多次实现。
 - 异步机制，微服务往往使用异步编程、消息与并行机制，会引入新的故障点。
 - 其他的像不可变基础设施、管理难度、微服务架构的推进等。

SOA ARCHITECTURE



Services/APIs



Application



Application Server



Managed Runtime Environment



Operating System



Hypervisor



Storage



Network

MICROSERVICES

Services/APIs



Managed Runtime Environment



Operating System



Hypervisor



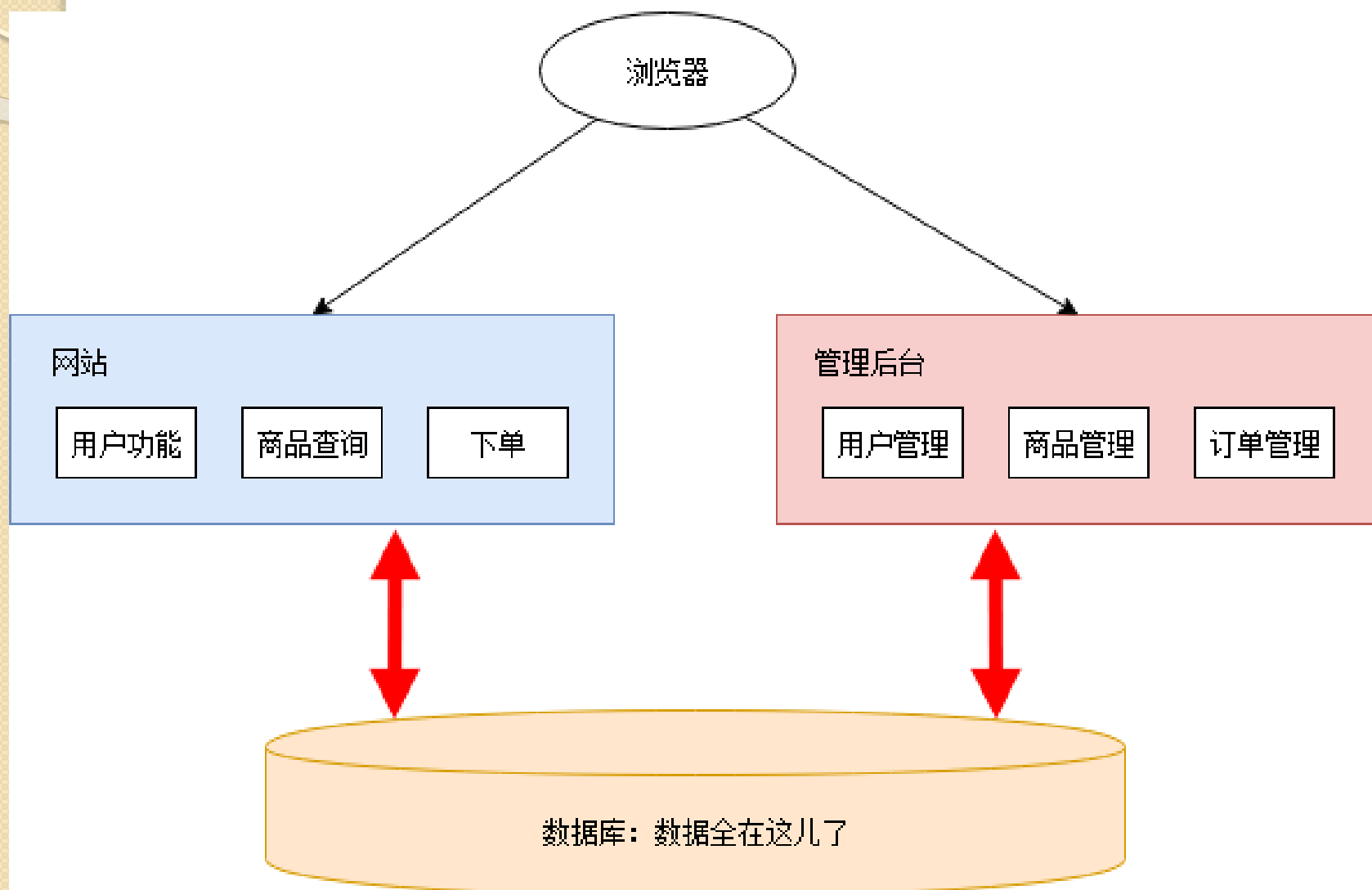
Storage



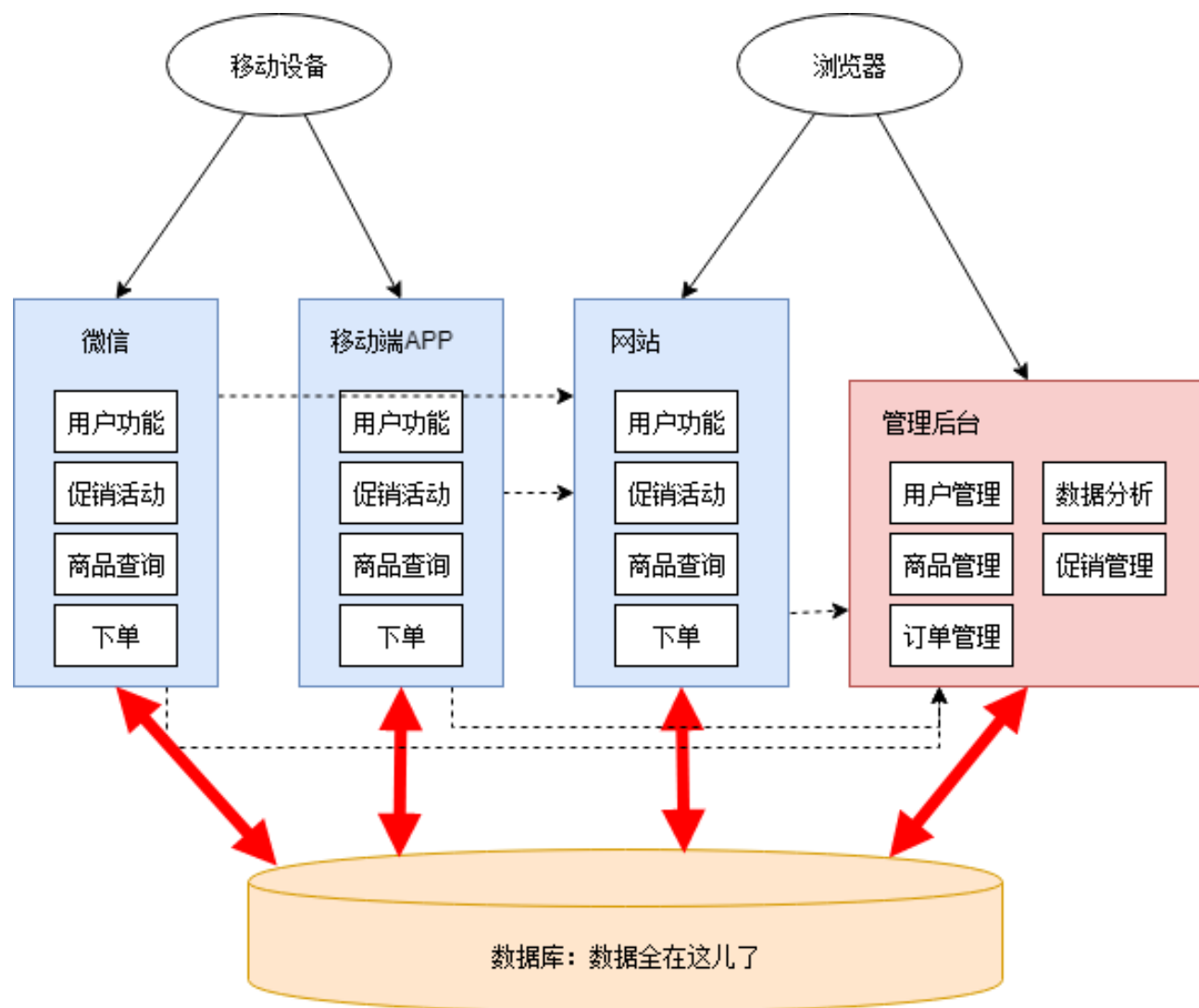
Network

功能	SOA	微服务
组件大小	大块业务逻辑	单独任务或小块业务逻辑
耦合	通常松耦合	总是松耦合
公司架构	任何类型	小型、专注于功能交叉的团队
管理	着重中央管理	着重分散管理
目标	确保应用能够交互操作	执行新功能，快速拓展开发团队

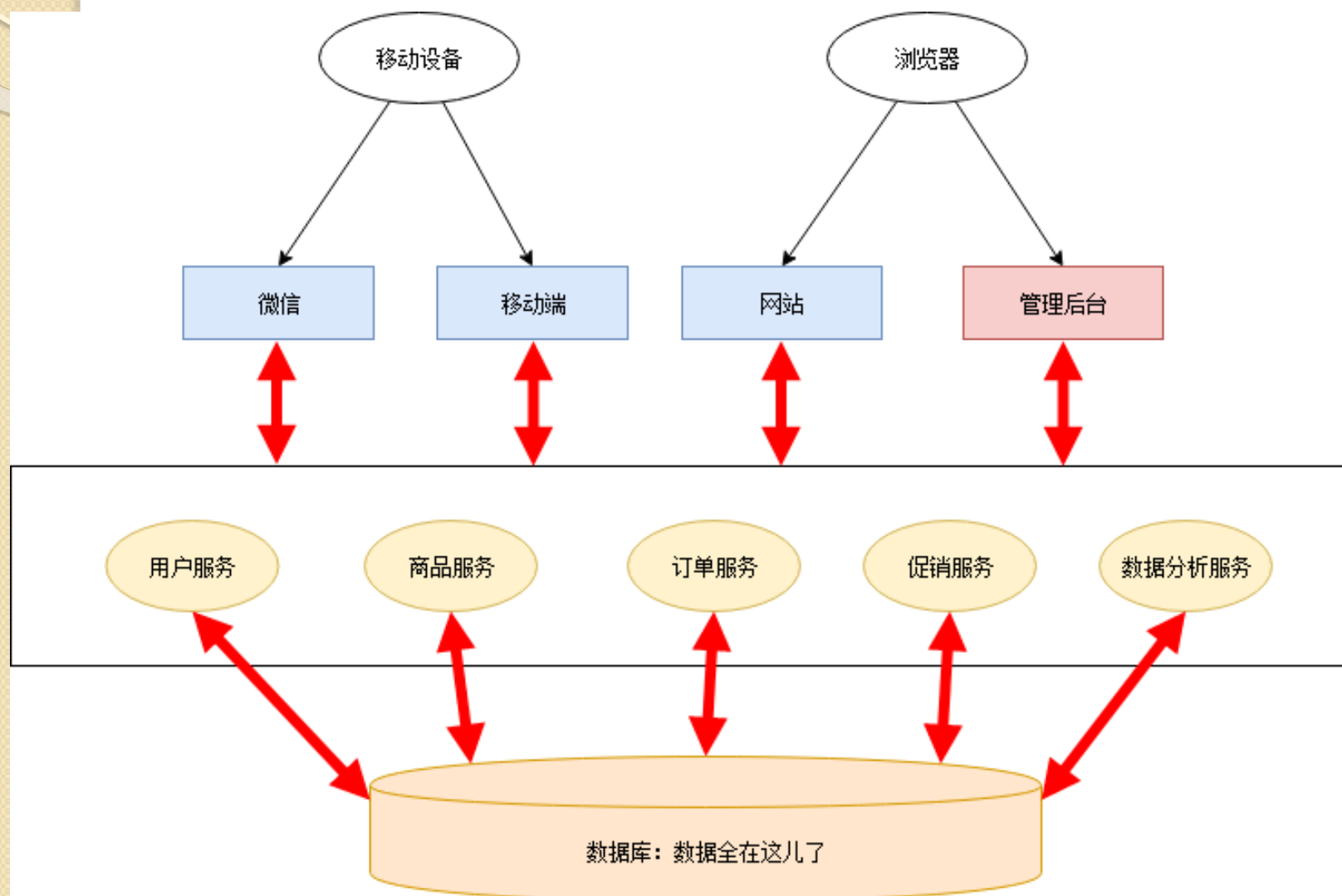
一个电商网站的架构演进

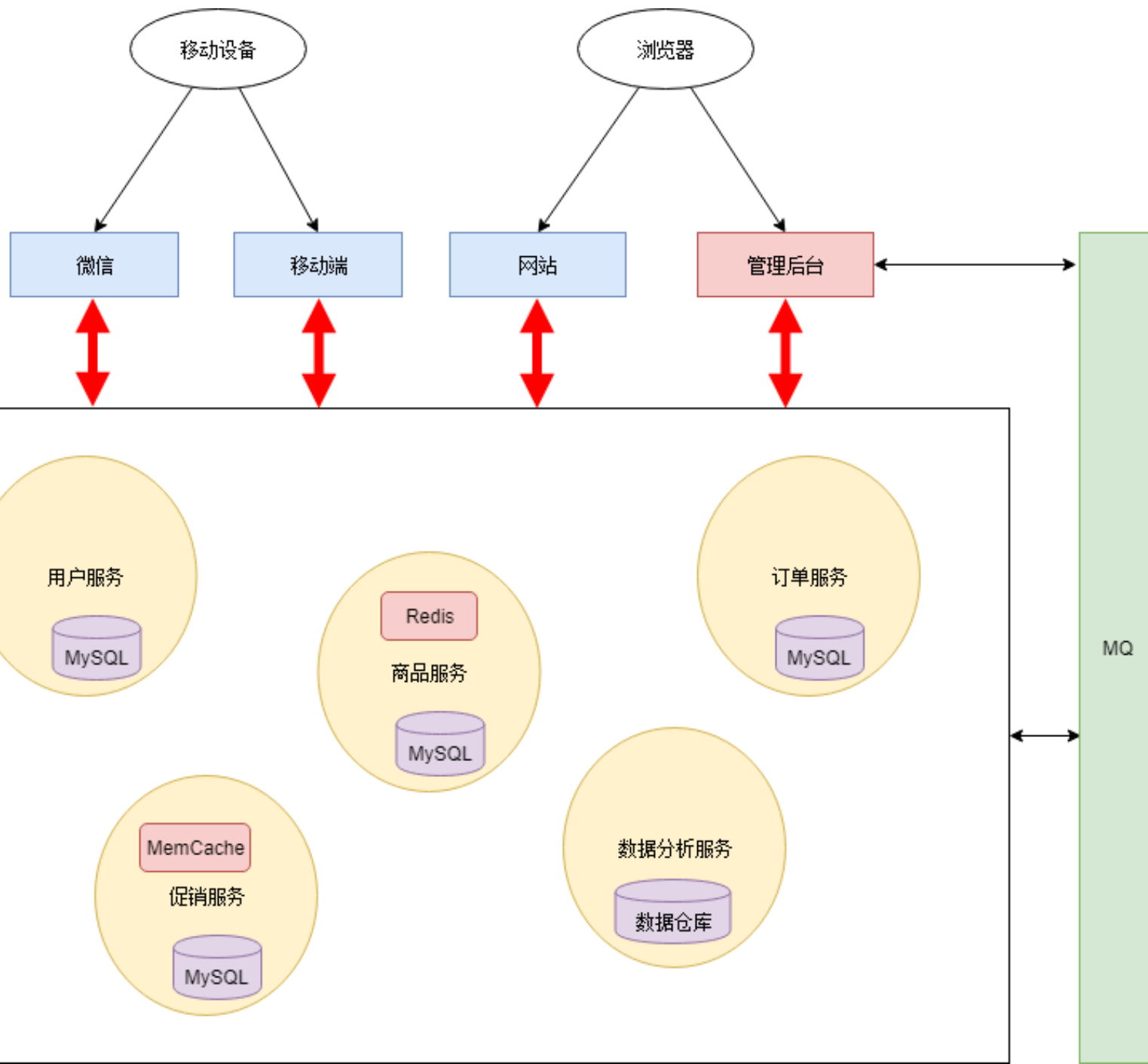


一个电商网站的架构演进



一个电商网站的架构演进





微信

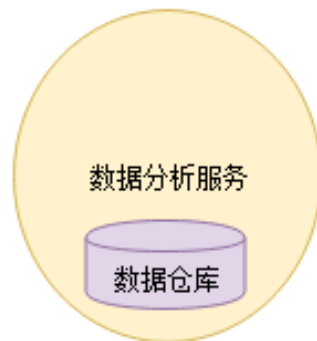
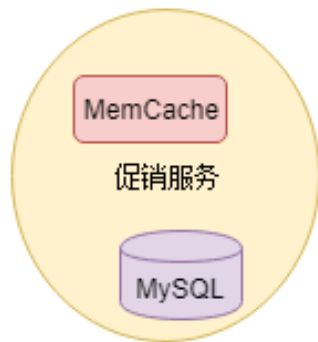
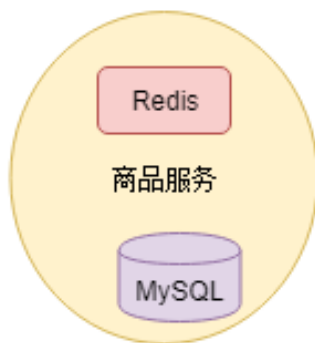
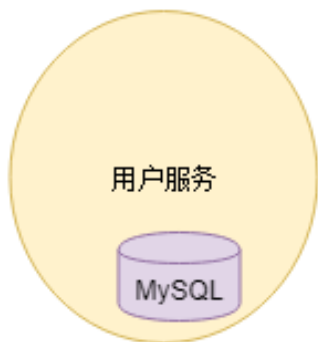
移动端

网站

管理后台



网关



4.3.17 正交架构风格

- 正交软件架构(Orthogonal Software Architecture) 由组织层(Layer)和线索(Thread)的组件(Component)构成。
 - 层是由一组具有相同抽象级别(Level of Abstraction)的组件构成。
 - 线索是子系统的特例，它是由完成不同层次功能的组件组成(通过相互调用来关联)，每一条线索完成整个系统中相对独立的一部分功能。
 - 如果线索是相互独立的，即不同线索中的组件之间没有相互调用，那么这个结构就是完全正交的。

4.3.17 正交架构风格

- 正交软件架构是一种以垂直线索组件族为基础的层次化结构
- 其基本思想是把应用系统的结构按功能的正交相关性，垂直分割为若干个线索(子系统)，线索又分为几个层次，每个线索由多个具有不同层次功能和不同抽象级别的组件构成。

4.3.17 正交架构风格

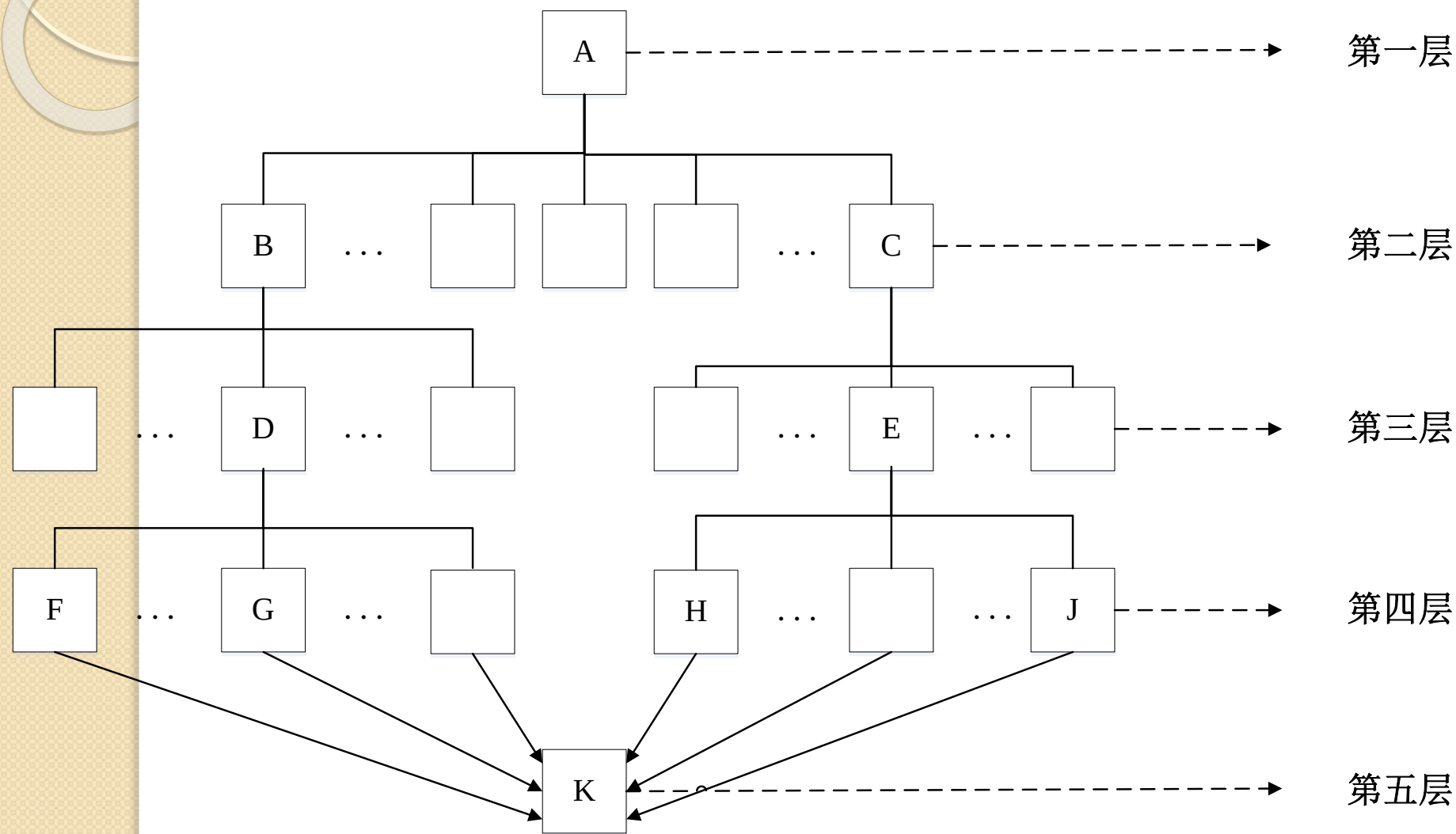


图4.43 正交软件架构

4.3.17 正交架构风格

- 特点：
 - (1) 由完成不同功能的 $n(n>1)$ 个线索(子系统)组成；
 - (2) 系统具有 $m(m > 1)$ 个不同抽象级别的层；
 - (3) 线索之间是相互独立的(正交的)；
 - (4) 系统有一个公共驱动层(一般为最高层)和公共数据结构(一般为最低层)。

4.3.17 正交架构风格

- 优点

- (1) 结构清晰，易于理解。由于线索功能相互独立，组件的位置可以清楚地说明它所实现的抽象层次和负担的功能。
- (2) 易修改，可维护性强。由于线索之间是相互独立的，所以对一个线索的修改不会影响到其他线索。
- (3) 可移植性强，重用粒度大。因为正交结构可以为一个领域内的所有应用程序所共享，这些软件有着相同或类似的层次和线索，可以实现架构级的重用。

4.3.17 正交架构风格

- 缺点
 - 在实际应用中，并不是所有软件系统都能完全正交化，或者有时完全正交化的成本太高。因此，在进行应用项目的软件架构设计时，必须反复权衡进一步正交化的额外开销与所得到的更好的性能之间的关系。

4.3.17 正交架构风格

- 应用实例：汽修服务管理系统

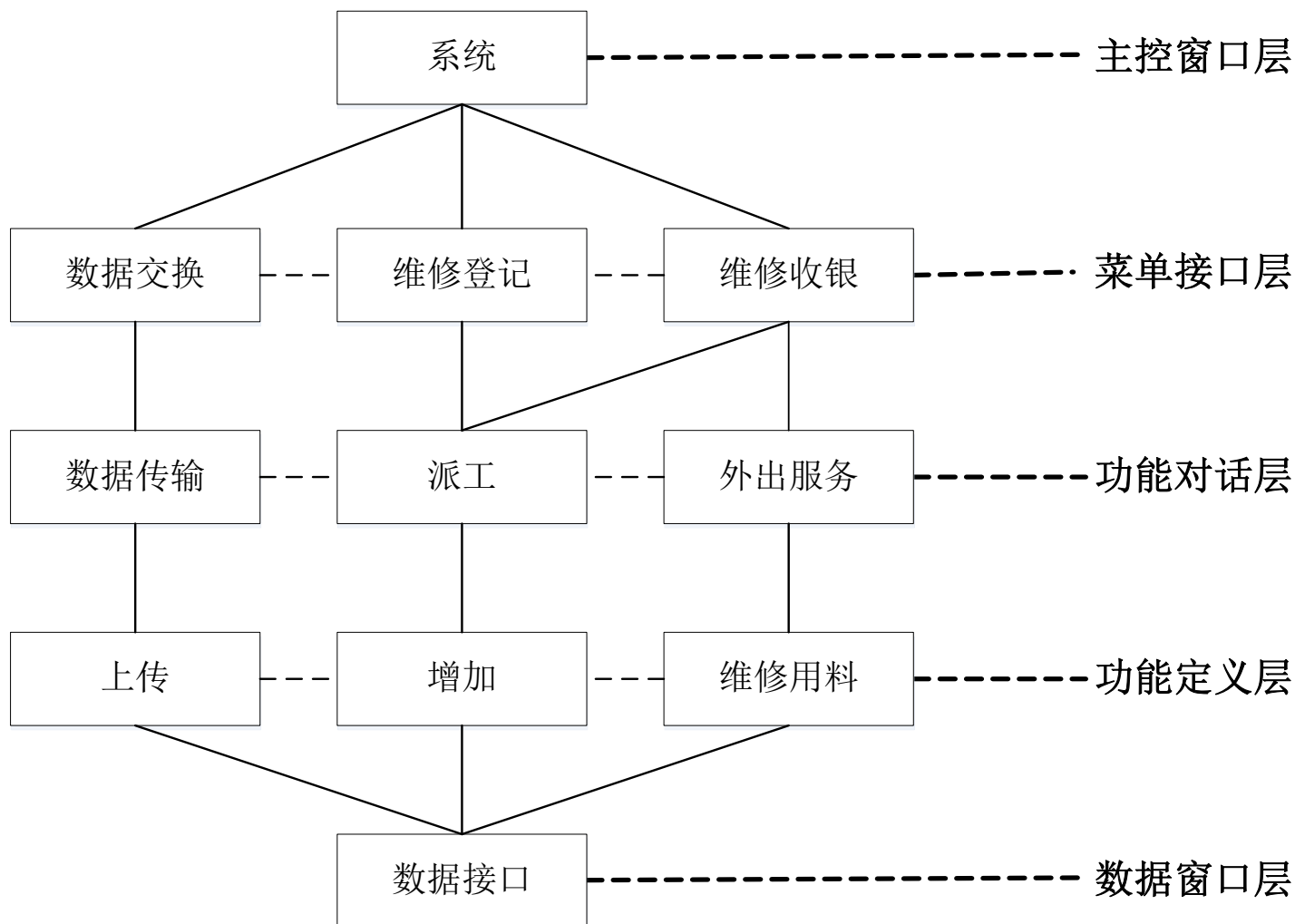


图4.44汽修服务管理系统的框架结构

4.3.18 异构风格

- 在设计软件系统时，从不同角度来观察和思考问题，会对架构风格的选择产生影响。
- 每一种架构风格都有不同的特点，适用于不同的应用问题，因此，架构风格的选择是多样化的和复杂的。
- 在实际应用中，各种软件架构并不是独立存在的，在一个系统中，往往会有多种架构共存和相互融合，形成更复杂的框架结构，即异构架构。

4.3.18 异构风格

- 异构架构是几种风格的组合，组合方式可能有如下几种：
 - （1）使用层次结构。一个系统组件被组织成某种架构风格，但它的内部结构可能是另一种完全不同的风格。
 - （2）允许单一组件使用复合的连接件。

4.3.18 异构风格

- 优点

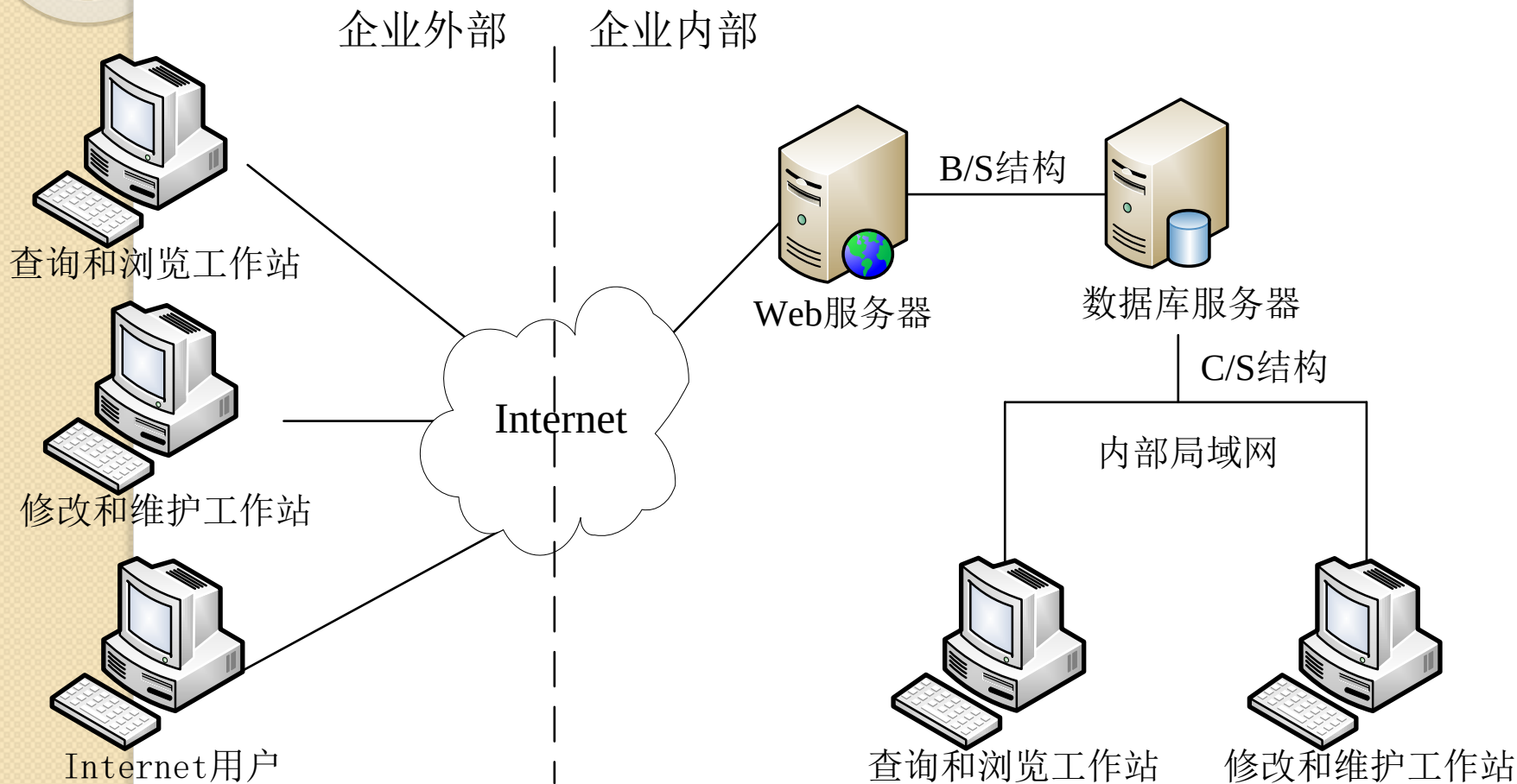
- (1) 选择异构架构风格，可以实现遗留代码的重用。
- (2) 在某一单位中，规定了共享软件包和某些标准，但仍会存在解释和表示习惯上的不同。选择异构架构风格，可以解决这一问题。

- 缺点

- 不同风格之间的兼容问题有时很难解决

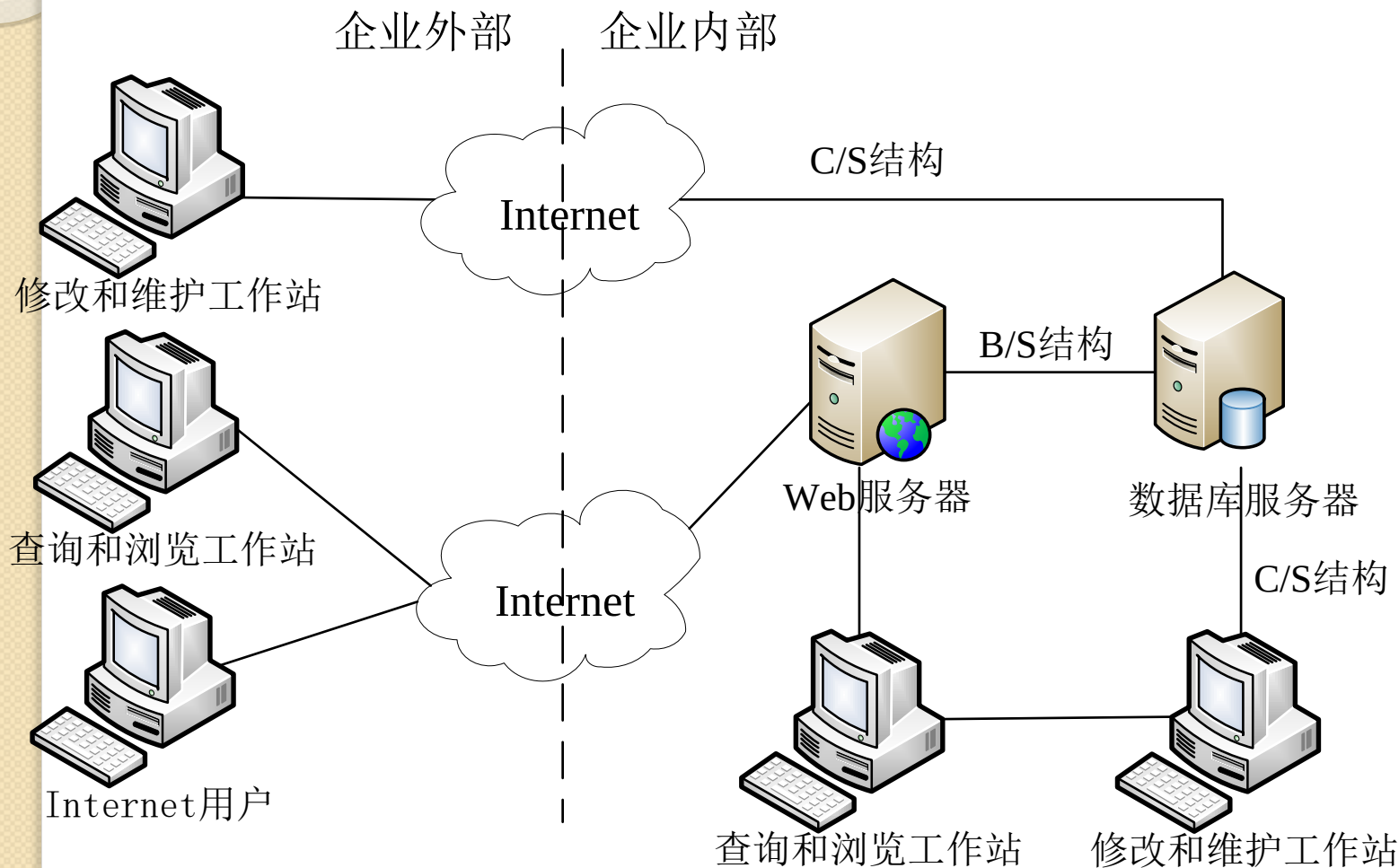
4.3.18 异构风格

应用实例：B/S结构和C/S结构组合——“内外有别”模型



4.3.18 异构风格

应用实例：B/S结构和C/S结构组合——“查改有别”模型



4.3.18 异构风格

- 应用实例：供电管理系统——B/S结构和C/S结构组合

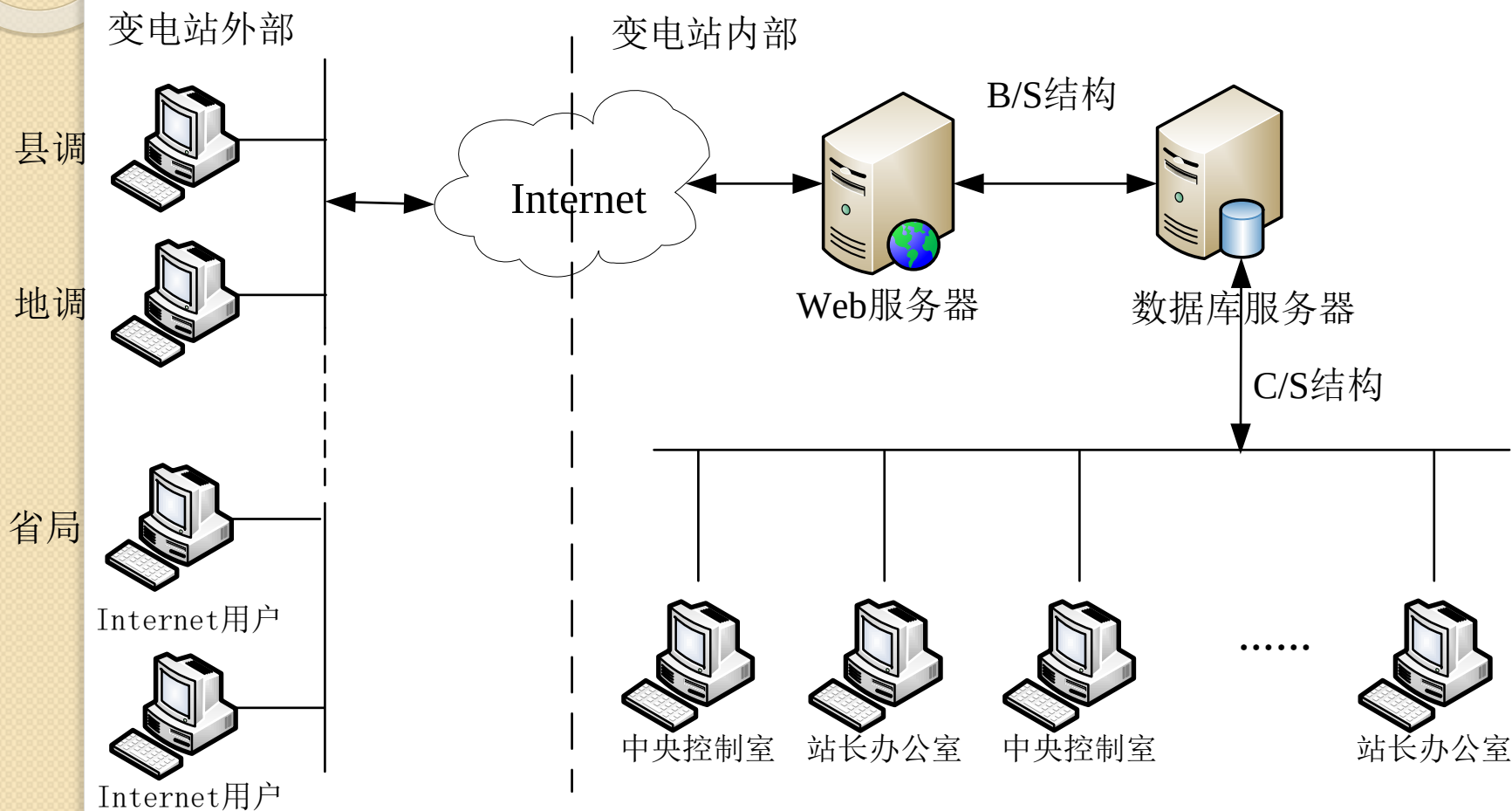


图4.47供电管理系统的架构

4.3.19 基于层次消息总线的架构风格 (JB/HMB风格)

- 北大杨芙清院士等提出；
- 以青鸟软件生产线的实践为背景，提出了基于层次消息总线的软件架构(jade bird hierarchical message bus based style)

4.3.19 基于层次消息总线的架构风格（JB/HMB风格）

- 背景：

- 网络和分布式构件技术日趋成熟，具有分布和并发特点的软件系统成为普遍需求；
- 基于事件驱动的编程模式在图形用户界面程序设计中得到广泛应用；
- 硬件总线体系结构具有良好的扩展性和适应性。

4.3.19 基于层次消息总线的架构风格 (JB/HMB风格)

- JB/HMB风格基于**层次消息总线**、支持组件的**分布**和**并发**，组件之间通过消息总线进行通讯。
- 消息总线是系统的连接件，负责消息的分派、传递和过滤以及处理结果的返回。
- 各个组件挂接在消息总线上，向总线登记感兴趣的**消息类型**。
- 不要求各个构件具有相同的地址空间或局限在同台机器上，可较好地描述分布式并发系统。

4.3.19 基于层次消息总线的架构风格 (JB/HMB风格)

- 基本思想

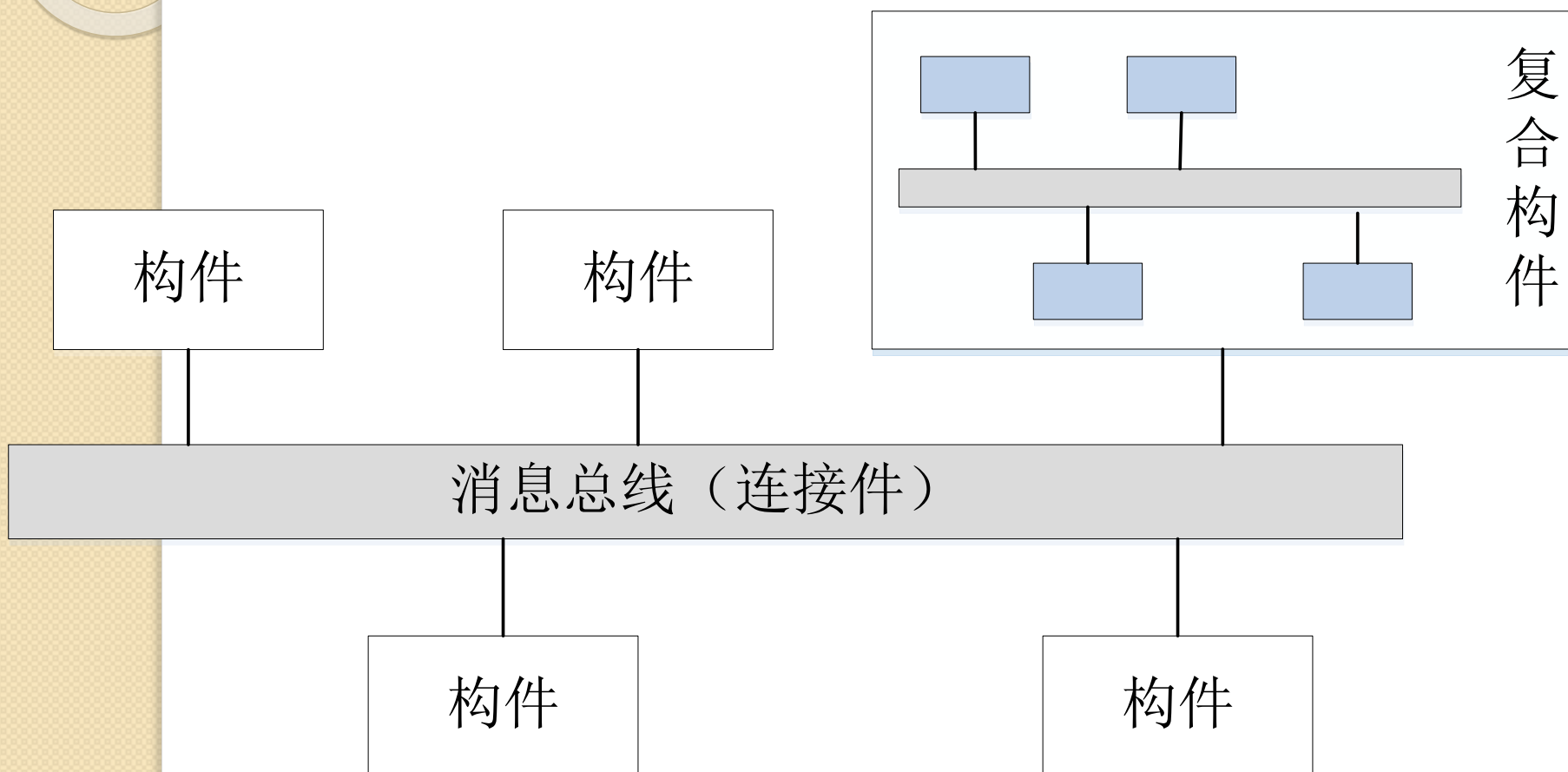


图4.48基于层次消息总线的架构风格

4.3.19 基于层次消息总线的架构风格

- 优点
 - (1) JB/HMB风格的构件接口是一种基于消息的互联接口，可以较好地支持架构设计。降低了构件之间的耦合性，增强了构件的重用性。
 - (2) JB/HMB风格支持运行时系统演化，主要体现在可动态增加和删除构件，动态改变构件所响应的消息以及消息过滤这三个方面。
- 缺点是重用要求高，可重用性差。

4.3.19 基于层次消息总线的架构风格

应用实例：青鸟工程软件开发过程

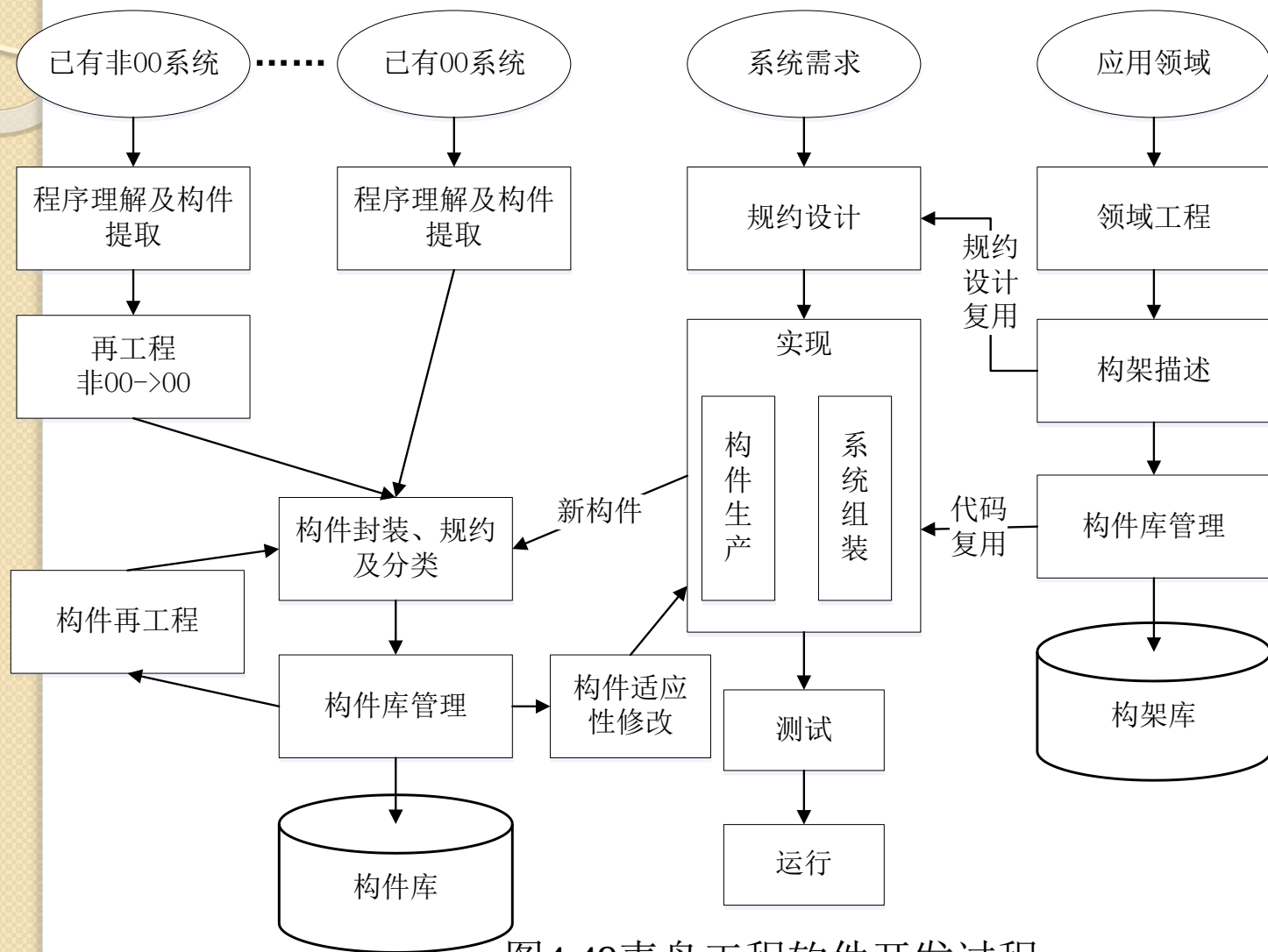


图4.49青鸟工程软件开发过程

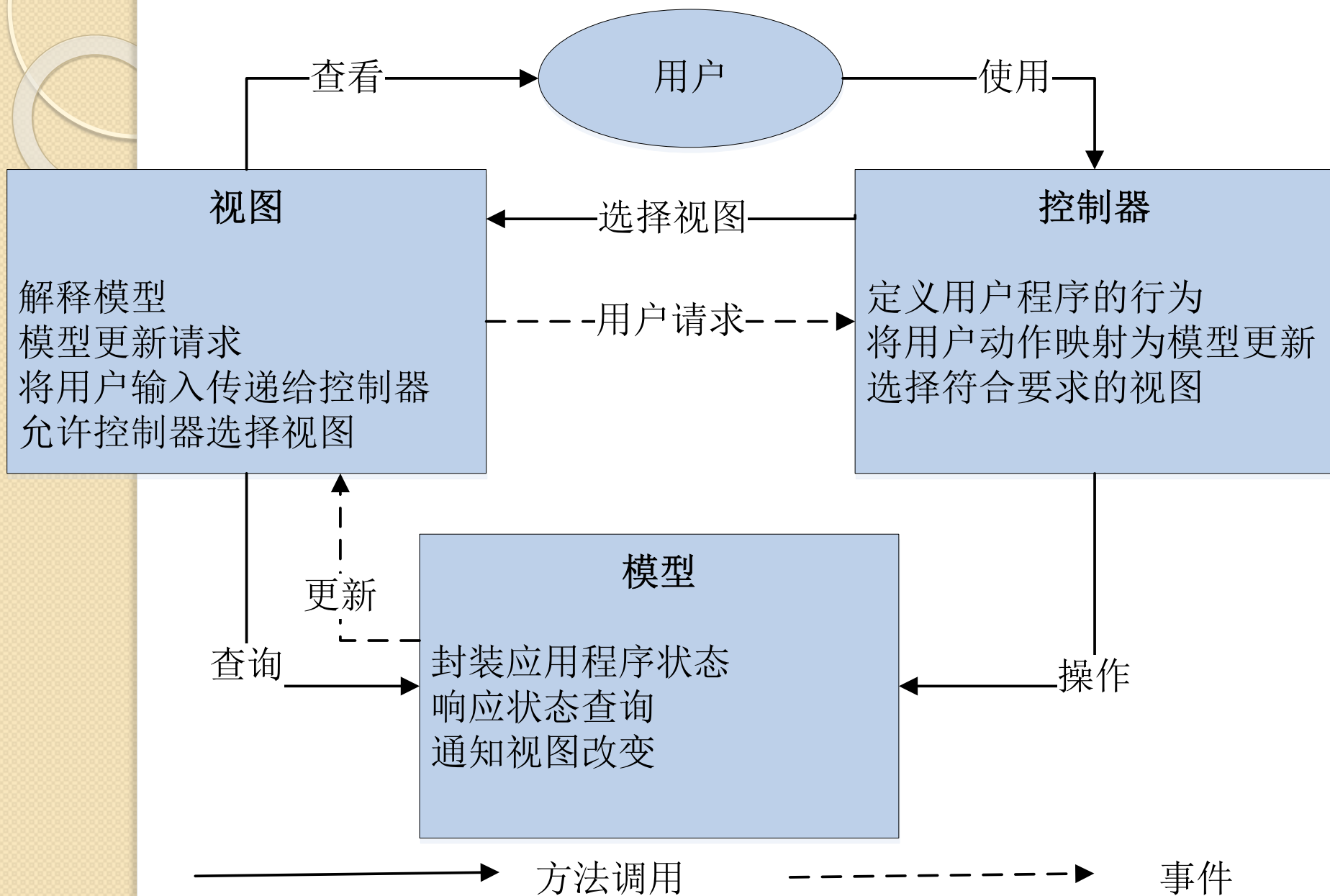
4.3.20 模型-视图-控制器风格

- 模型-视图-控制器风格 (Model-View-Controller, MVC) 主要是针对编程语言Smalltalk 80所提出的一种软件设计模式。
- MVC被广泛的应用于用户交互程序的设计中。

4.3.20 模型-视图-控制器风格

- MVC结构主要包括模型、视图和控制器三部分。
 - (1) 模型(Model, M): 模型是应用程序的核心, 它封装了问题的核心数据、逻辑关系和计算功能, 提供了处理问题的操作过程。
 - (2) 视图(View, V): 视图是模型的表示, 提供了交互界面, 为用户显示模型信息。
 - (3) 控制器(Controller, C): 控制器负责处理用户与系统之间的交互, 为用户提供操作接口。

4.3.20 模型-视图-控制器风格



4.3.20 模型-视图-控制器风格

- 优点

- (1) 多个视图与一个模型相对应。变化——传播机制确保了所有相关视图都能够及时地获取模型变化信息，从而使所有视图和控制器同步，便于维护。
- (2) 具有良好的移植性。由于模型独立于视图，因此可以方便的实现不同部分的移植。
- (3) 系统被分割为三个独立的部分，当功能发生变化时，改变其中的一个部分就能满足要求。

4.3.20 模型-视图-控制器风格

- 缺点

- (1) 增加了系统设计和运行复杂性。
- (2) 视图与控制器连接过于紧密，妨碍了二者的独立重用。
- (3) 视图访问模型的效率比较低。由于模型具有不同的操作接口，因此视图需要多次访问模型才能获得足够的数据。
- (4) 频繁访问未变化的数据，也将降低系统的性能。

4.3.20 模型-视图-控制器风格

- 应用实例

- 目前比较好的MVC

- 老牌的有Struts，Webwork。
- 新兴的MVC框架有Spring MVC，Tapestry，JSF等。
- 还有，如Dinamica，VRaptor等

- 这些框架都提供了较好的层次分隔能力。在实现良好的MVC分隔的基础上，提供一些现成的辅助类库，同时也促进了生产效率的提高。

4.2 软件架构风格的分类

- 根据上述架构风格分类发现，架构风格是基于不同的视角或层次抽象出来的。不同风格之间常常具有交叉现象。
 - 主程序和子程序风格以及面向对象风格几乎是目前一切软件构件、连接件的设计和实现基础；
 - 分层系统的层次结构风格几乎是一切复杂系统的基本结构方法；
 - 事件系统是一种受操作系统管理控制的部件连接方式；
 - 仓库是部件的设计结构；
 - 管道/过滤器、客户/服务器、解释器等则是特殊的系统结构方法。
 - 即：对于同一种软件架构，从不同的视角和层次可能抽象出不同的架构风格。

4.3 典型的软件架构风格

- 1.管道过滤器风格
- 2.主程序/子程序风格
- 3.面向对象风格
- 4.层次化风格
- 5.事件驱动风格
- 6.解释器风格
- 7.基于规则的系统风格
- 8.仓库风格
- 9.黑板系统风格
- 10.C2风格
- 11.客户机/服务器风格
- 12.浏览器/服务器风格
- 13.平台/插件风格
- 14.面向Agent风格
- 15.面向方面软件架构风格
- 16.面向服务架构风格
- 17.正交架构风格
- 18.异构风格
- 19.基于层次消息总线的架构风格
- 20.模型-视图-控制器风格

4.4 软件架构模式

- Dwayne E. Perry和Alexander L.Wolf从组件的角度给出了软件架构模式的定义：根据系统的结构组织定义了软件系统族，以及构成系统族的组件之间的关系。

4.4 软件架构模式

- 在一般意义上，大多数人认为模式即为风格
- Frank Buschmann 认为体系结构的模式与风格是有区别的，它们的区别和联系主要在于：
 - （1）体系结构风格主要描述应用系统的总体结构框架。
 - （2）架构风格相对独立。
 - （3）模式比架构风格更加面向问题。

4.4 软件架构模式

- 架构风格与模式的区别
 - 软件架构风格是对软件架构整体方案的展现形式，反映的是整体方案实施之后的效果；
 - 软件架构模式仍然是软件设计模式的一种，也是一种问题解决方案对（problem-solution pair），反映的是针对某个需求问题，至今为止最佳的解决方案是什么。
 - 虽然软件架构风格也与解决方案有关，但它并没有强调一定是最佳方案，而架构模式具有这种潜在要求。

补充：软件架构 & 软件框架

- 软件架构可以看做是一个软件系统从整体到部分的最高层次的划分。
- 根据组成派定义，软件架构主要反映系统由哪些部分组成，以及这些部分是如何组成的。
- 根据决策派定义，软件架构建造一个系统所作出的最高层次的、以后难以更改的，商业的和技术的决定。

补充： 软件架构 & 软件框架

- 什么是框架？
 - 框架，即framework。是某种应用的半成品，就是一组组件，供选用完成自己的系统。简单说就是使用别人搭好的舞台，你来做表演。而且，框架一般是成熟的，不断升级的软件。
- 框架与架构的区别？
 - 并无明确的定义，但一般从“层”的观点看，认为框架是底层的，接近系统的。软件开发者在其上构建自己的软件架构，开发自己的运用程序。

补充：为什么要用框架

- 因为软件系统发展到今天已经很复杂了，特别是服务器端软件，涉及到的知识，内容，问题太多。在某些方面使用成熟的框架，可以避免重复做已有的基础工作，而只需要集中精力完成系统的业务逻辑设计。
- 框架一般是成熟，稳健的，可以处理系统很多细节问题，比如，事物物理，安全性，数据流控制等问题。
- 框架一般都经过很多人使用，所以结构很好，所以扩展性也很好，而且它是不断升级的，使用框架的开发者可以直接享受别人升级代码带来的好处。
- 框架一般处在低层应用平台（如J2EE）和高层业务逻辑之间的中间层。

补充：目前学生进行B/S开发常用的技术框架

- 前端： JQuery、 Javascript、 HTML、 CSS
- 后端： java、 springboot、 mybatis、 elastic-job
- 数据库： mysql

SpringBoot2 + Vue + Redis的企业级聚合支付系统

Jeepay计全支付系统架构



练习1：这可以是哪个风格？

- **微核架构（microkernel architecture）** 指的是软件的内核相对较小，通常只包含系统运行的最小功能。其他构件则是互相独立的，它们之间的通信，应该减少到最低，避免出现互相依赖的问题。



- A. 平台/插件风格
- B. 仓库风格
- C. 事件驱动风格
- D. 基于Agent的风格

练习2：这可以是哪些风格？

- **溯源模式**，不会将系统当前状态存储在数据库中，而是将时间存储其中。例如 customer name 发生变化时，该值不会存储在“name”列中，我们会存储一个“NameChanged”事件。
- 当我们需要检索模型时，我们将检索其存储的所有事件并在新对象上重新应用，即 rehydrating an object。



◦ A. 仓库风格



◦ B. 事件驱动风格

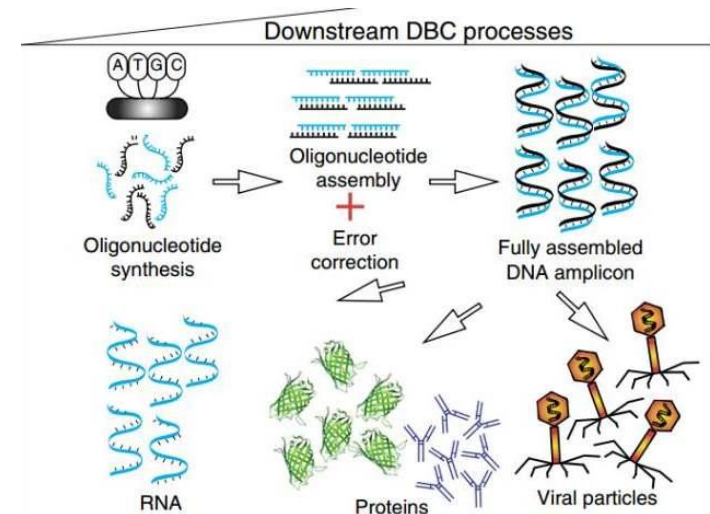
◦ C. 黑板

◦ D. 管道过滤器风格

练习3： 应该使用哪个风格？

- 根据核酸序列预测蛋白质结构的软件
- 就目前来说，蛋白质三维结构预测还没有一个通用的解决方案，对于蛋白结构预测只能得到相对精确的“近似解”。实现精准预测的目标尚未达到。
- 蛋白质结构由大量的局部组合构成，常用方法是Template Modeling，即根据已知模板推断结构信息。

黑板风格



练习4：这里使用了什么风格？

- 苹果为 iOS 12 带来了全新的通知功能，提供了一组用于监测和管理通知的管理工具。
- 来自同一款应用的多个通知，将在 iPhone 锁定在屏幕上以减少杂乱。轻触这一组，即可看到特定应用的详细通知列表。
- 点击分组通知旁边的 x，即可一次清理掉所有这些通知。或者通过向左滑动，也可触发相同的操作。
- 如果想要查看特定 app（比如消息）的所有传入通知，请点选“自动”、“按应用分组”、或者“关闭”。



面向服务、基于层次消息结构的风格

练习5：这里使用了什么风格？

- 在企业内部，有许多不同的网站，进入每个网站，都需要身份验证，不仅浪费时间而且容易遗忘代码，另外，网站维护人员对各种服务需要建立相应的用户认证与信息管理系统，分布于个服务器中的用户数据不仅浪费维护人员的时间，而且过于分散的用户数据不利于统计和管理。用户的需求和管理要求促使用户趋于统一，产生了统一认证。
- 应该采用什么风格解决统一认证问题？

面向方面